Instance-Level Recognition

Lecture notes accompanying Assignment 1
Object Recognition and Computer Vision

Study notes

Abstract

These notes collect the mathematical and algorithmic background needed for the first assignment on instance-level recognition: scale-space feature detection, the SIFT detector and descriptor, descriptor matching and Lowe's ratio test, planar geometric transformations, homogeneous least squares through the SVD, robust estimation with RANSAC, image warping with bilinear interpolation, and large-scale retrieval with bags of visual words, KD-trees and TF–IDF weighting. They explain *why* each tool works and how it is used, with proofs where they are short and illuminating. They deliberately do *not* contain the solutions of the assignment: whenever a task asks you to derive or implement something, the notes give you the concepts, an analogous worked example on a different problem, and the pitfalls to watch for, but leave the actual derivation and code to you.

Contents

1 Overview and roadmap **3**
- 1.1 The problem . . . 3
- 1.2 Where to find what . . . 3
- 1.3 Notation and conventions . . . 4

2 Mathematical preliminaries **4**
- 2.1 Convolution and Gaussian filtering . . . 4
- 2.2 Image gradients . . . 5
- 2.3 Transformations acting on images: invariance and covariance . . . 5

3 Scale space and blob detection **6**
- 3.1 Why we need scale . . . 6
- 3.2 Scale covariance of the scale space . . . 6
- 3.3 The Laplacian of Gaussian as a blob detector . . . 7
- 3.4 Difference of Gaussians . . . 7

4 SIFT: the Scale-Invariant Feature Transform **8**
- 4.1 The detector . . . 8
- 4.2 Frames as similarity transformations . . . 10
- 4.3 Covariance of the detector, invariance of the descriptor . . . 11
- 4.4 Repeatability . . . 11
- 4.5 The descriptor . . . 11
- 4.6 Properties of the SIFT descriptor . . . 12

5 Matching descriptors **14**
- 5.1 Nearest-neighbour matching . . . 14
- 5.2 Computing all distances efficiently . . . 14
- 5.3 Lowe's ratio test . . . 15
- 5.4 Evaluating a set of matches . . . 16

6 Planar geometric transformations **16**
6.1 Euclidean, similarity and affine transformations . . . 16
6.2 Homogeneous coordinates and the projective plane . . . 17
6.3 The cross product as a linear map . . . 18
6.4 Degrees of freedom and minimal samples . . . 19

7 Homogeneous linear systems and the SVD **19**
7.1 The problem . . . 19
7.2 Solution by the singular value decomposition . . . 20
7.3 The Direct Linear Transform . . . 20
7.4 Algebraic error and data normalisation . . . 21

8 Robust estimation with RANSAC **22**
8.1 Why least squares is not enough . . . 22
8.2 The algorithm . . . 22
8.3 How many iterations? . . . 23
8.4 Choosing the threshold . . . 24
8.5 Refinement and variants . . . 24
8.6 A generic implementation . . . 24

9 Image warping, interpolation and compositing **25**
9.1 Forward versus inverse warping . . . 25
9.2 Interpolation . . . 25
9.3 Alpha compositing . . . 27

10 Large-scale retrieval with bags of visual words **28**
10.1 Why pairwise matching does not scale . . . 28
10.2 Visual vocabulary: k-means clustering . . . 28
10.3 Fast nearest-word search: KD-trees . . . 29
10.4 The bag-of-visual-words histogram . . . 30
10.5 TF–IDF weighting and normalisation . . . 31
10.6 Similarity scores . . . 31
10.7 Inverted files . . . 32
10.8 Evaluating a ranking . . . 32

11 Geometric re-ranking of a shortlist **32**
11.1 Why re-rank? . . . 32
11.2 Tentative matches from visual words . . . 32
11.3 A simpler transformation model . . . 33
11.4 What to look for in the analysis . . . 33

A NumPy and SciPy toolbox **33**
A.1 Shapes, axes and broadcasting . . . 33
A.2 Finding minima and the k smallest values . . . 34
A.3 Selecting with masks and indices . . . 34
A.4 Safe division . . . 34
A.5 Angles . . . 34
A.6 Random numbers . . . 35
A.7 Data coming from MATLAB . . . 35
A.8 Image types . . . 35

1 Overview and roadmap

1.1 The problem

Instance-level recognition asks whether a *specific* object or scene (this building, this painting, this poster) is visible in an image, and where. It differs from *category-level* recognition ("is there *a* building?"): we are not generalising over a class, we are re-identifying one particular physical object across changes of

- **viewpoint** (translation, rotation, scaling, perspective distortion),
- **illumination** (brightness and contrast changes, shadows),
- **occlusion and clutter** (only part of the object is visible, surrounded by other things).

The classical answer, which this assignment follows, is built on *local features*. Instead of comparing two images globally (which fails as soon as part of the object is occluded or the viewpoint changes), we extract many small, distinctive, repeatable regions from each image, describe each region by a vector, match the vectors, and finally check that the matches agree with a geometric transformation between the two images.

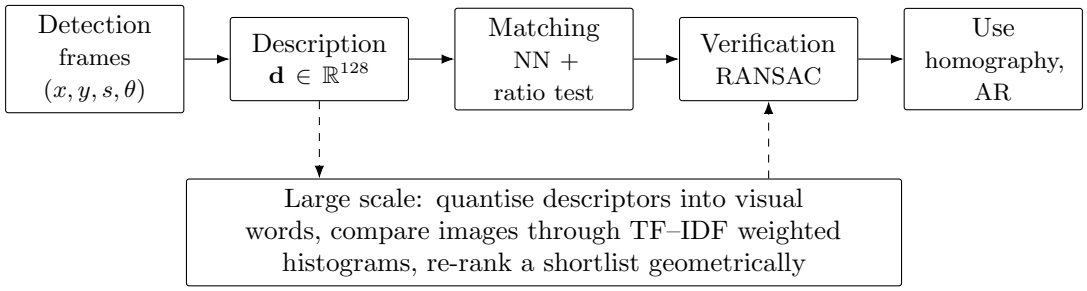


Figure 1: The pipeline of the assignment. Part 1 follows the top row for a pair of images; Part 2 replaces exhaustive descriptor matching by a compact global representation that scales to large databases.

1.2 Where to find what

The following table maps each task of the assignment to the parts of these notes that contain the background you need. The notes never contain the answer itself.

Task	Topic	Sections
1.1	SIFT frames on a pair of natural images; repeatability	3, 4.1, 4.3
1.2	Effect of the peak (contrast) threshold	4.1.3, 4.4
1.3	SIFT descriptor versus raw patches	4.5, 4.6
1.4	Nearest-neighbour matching	5, 5.2, App. A
1.5	Lowe's ratio test	5.3
1.6	Similarity from one frame correspondence	4.2, 6.1
1.7	RANSAC with a similarity model	8
1.8	Linear equations for a homography	6.2, 6.3, 7.3
1.9	Degrees of freedom, minimal sample, degeneracy	6.4, 7.3
1.10	Homography by homogeneous least squares	7.2, 7.4
1.11	Inverse warping, bilinear interpolation, compositing	9
2.1	KD-tree word assignment	10.2, 10.3
2.2	Bag-of-visual-words histogram	10.4
2.3	Retrieval scores	10.5, 10.6
2.4	Geometric re-ranking (optional)	11

1.3 Notation and conventions

Notation 1.1. A grayscale image is a function $I : \Omega \subset \mathbb{R}^2 \to \mathbb{R}$, sampled on the pixel grid. Points are column vectors $\mathbf{x} = (x, y)^\top$. In image coordinates (NumPy, OpenCV, Matplotlib's `imshow`) the origin is the *top-left* corner, x grows to the *right* (columns) and y grows *downwards* (rows). Consequently an image array is indexed as `I[y, x]`, i.e. `I[row, column]`: the order is the *opposite* of the order in $\mathbf{x} = (x, y)^\top$.

Notation 1.2. Bold lower-case letters are vectors, upper-case letters matrices. $\|\cdot\|$ is the Euclidean norm, $\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^\top \mathbf{b}$ the dot product. $R(\theta)$ denotes the 2×2 rotation matrix

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

For a vector $\mathbf{a} \in \mathbb{R}^3$, $[\mathbf{a}]_\times$ denotes the skew-symmetric matrix of the cross product (Section 6.3). The symbol $\sim$ between homogeneous vectors means "equal up to a non-zero scale factor".

Remark 1.3 (Orientation of angles). Because the y axis points down, a positive angle θ in image coordinates corresponds to a *clockwise* rotation on screen. This is harmless as long as *every* angle you manipulate (keypoint orientations, the angle in $R(\theta)$, `np.arctan2`) is expressed in the same coordinate system. OpenCV reports keypoint orientations in degrees, measured in this same image coordinate system; the notebook's helper converts them to radians.

2 Mathematical preliminaries

2.1 Convolution and Gaussian filtering

Definition 2.1 (Convolution)**.** For $f, g : \mathbb{R}^2 \to \mathbb{R}$ integrable, the convolution is

$$(f * g)(\mathbf{x}) = \int_{\mathbb{R}^2} f(\mathbf{x} - \mathbf{y})\, g(\mathbf{y})\, d\mathbf{y}.$$

On a discrete grid the integral becomes a sum over pixels.

Convolution is linear, commutative, associative and *commutes with translations*: if $(\tau_{\mathbf{t}} f)(\mathbf{x}) = f(\mathbf{x} - \mathbf{t})$ then $\tau_{\mathbf{t}}(f * g) = (\tau_{\mathbf{t}} f) * g$. It also commutes with differentiation: $\partial_x (f * g) = (\partial_x f) * g = f * (\partial_x g)$. The second property is what allows us to compute derivatives of a smoothed image by convolving with derivatives of the smoothing kernel.

Definition 2.2 (Gaussian kernel)**.** The isotropic two-dimensional Gaussian of standard deviation $\sigma > 0$ is

$$G_\sigma(\mathbf{x}) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{\|\mathbf{x}\|^2}{2\sigma^2} \right), \qquad \int_{\mathbb{R}^2} G_\sigma = 1.$$

Proposition 2.3 (Properties of the Gaussian)**.** *(i)* Separability: $G_\sigma(x, y) = g_\sigma(x)\, g_\sigma(y)$ *with* g_σ *the 1D Gaussian; hence a 2D Gaussian blur is two 1D blurs (cost* $O(k)$ *instead of* $O(k^2)$ *per pixel for a* $k \times k$ *kernel).*

(ii) Semigroup property: $G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2 + \sigma_2^2}}$. *Blurring an already blurred image is again a Gaussian blur, with variances adding up.*

(iii) Rotational invariance: $G_\sigma(R\mathbf{x}) = G_\sigma(\mathbf{x})$ *for every rotation* R.

(iv) Scaling: $G_\sigma(\mathbf{x}) = \frac{1}{\sigma^2} G_1(\mathbf{x}/\sigma)$.

(v) Heat equation: $\dfrac{\partial G_\sigma}{\partial \sigma} = \sigma\, \nabla^2 G_\sigma$, *where* $\nabla^2 = \partial_{xx} + \partial_{yy}$ *is the Laplacian.*

Proof. (i), (iii), (iv) are immediate from the formula. (ii) is the statement that the sum of two independent Gaussian random vectors with covariances $\sigma_1^2 I$ and $\sigma_2^2 I$ is Gaussian with covariance $(\sigma_1^2+\sigma_2^2)I$, since the density of a sum is the convolution of the densities. For (v), write $r^2 = \|\mathbf{x}\|^2$. Direct differentiation gives

$$\frac{\partial G_\sigma}{\partial \sigma} = \left(\frac{r^2}{\sigma^3} - \frac{2}{\sigma}\right) G_\sigma, \qquad \nabla^2 G_\sigma = \left(\frac{r^2}{\sigma^4} - \frac{2}{\sigma^2}\right) G_\sigma,$$

and the first expression is σ times the second. □

2.2 Image gradients

The gradient $\nabla I = (I_x, I_y)^\top$ points in the direction of steepest intensity increase. We write its *magnitude* and *orientation* as

$$m(\mathbf{x}) = \sqrt{I_x^2 + I_y^2}, \qquad \phi(\mathbf{x}) = \operatorname{atan2}(I_y, I_x) \in (-\pi, \pi].$$

On a discrete image the derivatives are approximated by finite differences, e.g. $I_x(x,y) \approx \frac{1}{2}\big(I(x+1,y) - I(x-1,y)\big)$, usually after Gaussian smoothing to reduce noise. Two elementary facts will be used repeatedly.

Proposition 2.4 (Behaviour of gradients)**.** *(i)* Affine intensity changes. *If $I'(\mathbf{x}) = aI(\mathbf{x}) + b$ with $a > 0$, then $\nabla I' = a\nabla I$: the orientation ϕ is unchanged and the magnitude is multiplied by a; the offset b disappears.*

(ii) Rotations. *If $I'(\mathbf{x}) = I(R^{-1}\mathbf{x})$ for a rotation $R = R(\alpha)$, then $\nabla I'(\mathbf{x}) = R\,\nabla I(R^{-1}\mathbf{x})$: magnitudes are preserved and orientations are shifted by α.*

Proof. (i) is linearity of differentiation. (ii) follows from the chain rule: $\nabla_{\mathbf{x}}[I(R^{-1}\mathbf{x})] = (R^{-1})^\top \nabla I(R^{-1}\mathbf{x}) = R\,\nabla I(R^{-1}\mathbf{x})$ because $R^{-1} = R^\top$. □

Property (i) is the reason gradient-based descriptors are robust to illumination, and property (ii) is the reason they can be made rotation invariant by expressing every orientation *relative* to a reference orientation.

2.3 Transformations acting on images: invariance and covariance

Let $\mathcal{G}$ be a group of transformations $T : \mathbb{R}^2 \to \mathbb{R}^2$ (for instance translations, rotations, similarities, …; see Section 6). A transformation T acts on an image by moving its content:

$$(T \cdot I)(\mathbf{x}) = I\big(T^{-1}(\mathbf{x})\big).$$

The inverse appears because the value of the new image at $\mathbf{x}$ is the value of the old image at the point that T sends to $\mathbf{x}$.

Definition 2.5 (Invariance and covariance)**.** Let Φ be a function computed from images.

- Φ is *invariant* to $\mathcal{G}$ if $\Phi(T \cdot I) = \Phi(I)$ for all $T \in \mathcal{G}$: the output does not change.
- Φ is *covariant* (or *equivariant*) to $\mathcal{G}$ if its output lives in a space on which $\mathcal{G}$ also acts and $\Phi(T \cdot I) = T \cdot \Phi(I)$: the output changes *in the same way* as the input.

Example 2.6. The set of local maxima of $|I * G_\sigma|$ is covariant to translations and rotations (the maxima move with the image) but is *not* covariant to scaling, because a fixed σ means a fixed physical size in pixels. The mean intensity of an image is invariant to translations (up to boundary effects) but not to contrast changes.

For local features we want a **covariant detector** (the detected regions follow the transformation, so that the same physical patch is found in both images) and an **invariant descriptor** (the vector computed on that patch does not change). Section 4.3 shows that the first property essentially implies the second, once the descriptor is computed in a normalised coordinate system attached to the detected region.

3 Scale space and blob detection

3.1 Why we need scale

A corner or a blob has no intrinsic size *in pixels*: the same window of a facade covers 40 pixels in a close-up and 10 pixels in a distant view. Any detector that operates at a single, fixed scale can therefore only be covariant to translations and rotations. To handle zoom we must analyse the image at *all* scales simultaneously and let the image itself tell us which scale is appropriate at each location. This is the idea of *scale space* and *automatic scale selection* [2].

Definition 3.1 (Gaussian scale space)**.** The (linear) scale space of an image I is the family of smoothed images

$$L(\mathbf{x}, \sigma) = (G_\sigma * I)(\mathbf{x}), \qquad \sigma > 0.$$

σ is the *scale* parameter: structures much smaller than σ are blurred away in $L(\cdot, \sigma)$.

Under natural requirements (linearity, shift invariance, rotational symmetry, and the requirement that smoothing never *creates* new local extrema in 1D), the Gaussian is the unique admissible smoothing kernel; we will simply take this as given. By the semigroup property (Proposition 2.3(ii)), $L(\cdot, \sigma_2)$ can be computed from $L(\cdot, \sigma_1)$ for $\sigma_2 > \sigma_1$ by a further blur of standard deviation $\sqrt{\sigma_2^2 - \sigma_1^2}$, which is how the scale space is computed incrementally in practice.

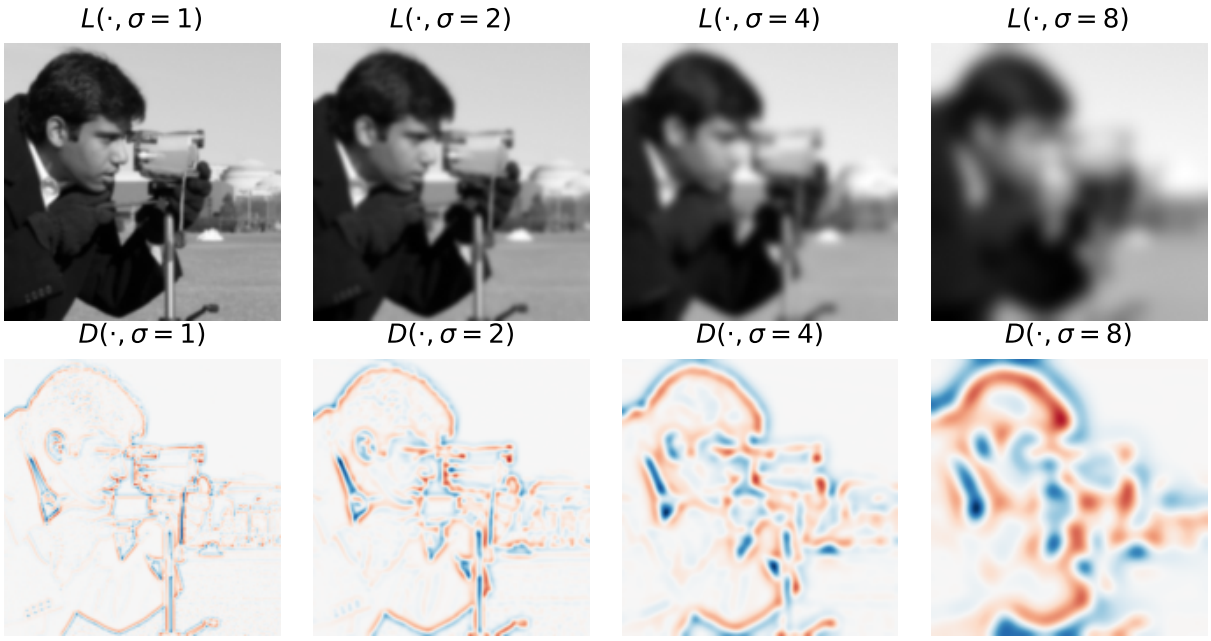


Figure 2: Top: Gaussian scale space $L(\cdot, \sigma)$ of an image crop. Bottom: the corresponding difference of Gaussians $D(\cdot, \sigma) = L(\cdot, k\sigma) - L(\cdot, \sigma)$ with $k = 2^{1/3}$ (red: positive, blue: negative). Each scale responds to structures of a different size; fine textures disappear as σ grows.

3.2 Scale covariance of the scale space

Proposition 3.2 (Scaling the image shifts the scale space)**.** *Let $c > 0$ and $I'(\mathbf{x}) = I(\mathbf{x}/c)$, i.e. I' is I magnified by c. Then*

$$L'(\mathbf{x}, c\sigma) = L(\mathbf{x}/c, \sigma) \qquad \text{for all } \mathbf{x}, \sigma.$$

Moreover, for the scale-normalised Laplacian $\sigma^2 \nabla^2 L$,

$$(c\sigma)^2 \, \nabla^2 L'(\mathbf{x}, c\sigma) = \sigma^2 \, \nabla^2 L(\mathbf{x}/c, \sigma).$$

Proof. With the change of variables $\mathbf{y} = c\mathbf{u}$,

$$L'(\mathbf{x}, c\sigma) = \int G_{c\sigma}(\mathbf{x} - \mathbf{y})\, I(\mathbf{y}/c)\, d\mathbf{y} = \int G_{c\sigma}\big(c(\mathbf{x}/c - \mathbf{u})\big)\, I(\mathbf{u})\, c^2\, d\mathbf{u}.$$

By Proposition 2.3(iv), $G_{c\sigma}(c\mathbf{z}) = \frac{1}{c^2\sigma^2} G_1(\mathbf{z}/\sigma) = \frac{1}{c^2} G_\sigma(\mathbf{z})$, so the integral equals $\int G_\sigma(\mathbf{x}/c - \mathbf{u}) I(\mathbf{u})\, d\mathbf{u} = L(\mathbf{x}/c, \sigma)$. Differentiating twice with respect to $\mathbf{x}$ brings out a factor $1/c^2$, which is exactly cancelled by the normalisation $(c\sigma)^2$ versus σ^2. □

Corollary 3.3. *If $(\mathbf{x}_0, \sigma_0)$ is a local extremum of $\sigma^2\nabla^2 L$ (jointly in space* and *scale), then $(c\mathbf{x}_0, c\sigma_0)$ is a local extremum of $\sigma^2\nabla^2 L'$. Hence detecting extrema of the normalised Laplacian over $(\mathbf{x}, \sigma)$ is* covariant to scaling*: both the position and the selected scale follow the zoom.*

The normalisation factor σ^2 is essential. Without it, derivative responses decay systematically with σ (the image becomes smoother), so the maximum over scales would always be at the smallest scale and would carry no information about the size of the structure.

3.3 The Laplacian of Gaussian as a blob detector

The operator $\nabla^2 G_\sigma$ ("Mexican hat", or rather its negative) is a centre–surround filter: it responds strongly to a dark blob on a bright background (positive response) or to a bright blob on a dark background (negative response) whose size matches σ.

Proposition 3.4 (Characteristic scale of a Gaussian blob)**.** *Let $I = 2\pi t\, G_{\sqrt{t}}$ be a bright Gaussian blob of variance t and unit peak value. At the centre of the blob, the normalised Laplacian response*

$$\sigma^2\nabla^2 L(\mathbf{0}, \sigma) = -\frac{2t\,\sigma^2}{(t + \sigma^2)^2}$$

attains its maximum absolute value at $\sigma^\star = \sqrt{t}$, and the maximum value $1/2$ does not depend on t.

Proof. By the semigroup property $L(\cdot, \sigma) = 2\pi t\, G_s$ with $s^2 = t + \sigma^2$. From the proof of Proposition 2.3(v), $\nabla^2 G_s(\mathbf{0}) = -\frac{2}{s^2} G_s(\mathbf{0}) = -\frac{1}{\pi s^4}$. Hence $\sigma^2\nabla^2 L(\mathbf{0}, \sigma) = -2t\sigma^2/(t + \sigma^2)^2$. Writing $u = \sigma^2$, the function $u \mapsto u/(t + u)^2$ has derivative $(t - u)/(t + u)^3$, which vanishes only at $u = t$, where it changes sign from positive to negative. The value there is $2t \cdot t/(2t)^2 = 1/2$. □

So the scale at which the response peaks *measures the size of the blob*, and the peak height is the same whatever that size is: this is automatic scale selection. For a uniform disk of radius r one obtains in the same way $\sigma^\star = r/\sqrt{2}$. Figure 3 illustrates the statement.

3.4 Difference of Gaussians

Computing $\nabla^2 L$ at many scales is expensive. Lowe [1] observed that the *difference of Gaussians* (DoG) is an excellent and cheap approximation.

Proposition 3.5 (DoG approximates the normalised LoG)**.** *For $k > 1$ close to 1,*

$$G_{k\sigma} - G_\sigma \;\approx\; (k-1)\,\sigma^2\nabla^2 G_\sigma, \qquad \textit{hence} \qquad D(\mathbf{x}, \sigma) \coloneqq L(\mathbf{x}, k\sigma) - L(\mathbf{x}, \sigma) \approx (k-1)\,\sigma^2\nabla^2 L(\mathbf{x}, \sigma).$$

Proof. By a first-order Taylor expansion in σ and the heat equation (Proposition 2.3(v)),

$$G_{k\sigma} - G_\sigma \approx (k\sigma - \sigma)\frac{\partial G_\sigma}{\partial \sigma} = (k - 1)\sigma \cdot \sigma\nabla^2 G_\sigma.$$

Convolving with I gives the second statement. □

Two consequences make DoG attractive. First, the factor $(k - 1)$ is the *same* at every scale if the scales are sampled geometrically ($\sigma_{i+1} = k\sigma_i$), so DoG is *automatically* scale-normalised and its extrema across scales can be compared directly. Second, it costs a single subtraction of two images that we compute anyway when building the scale space. Figure 4 compares the two kernels.

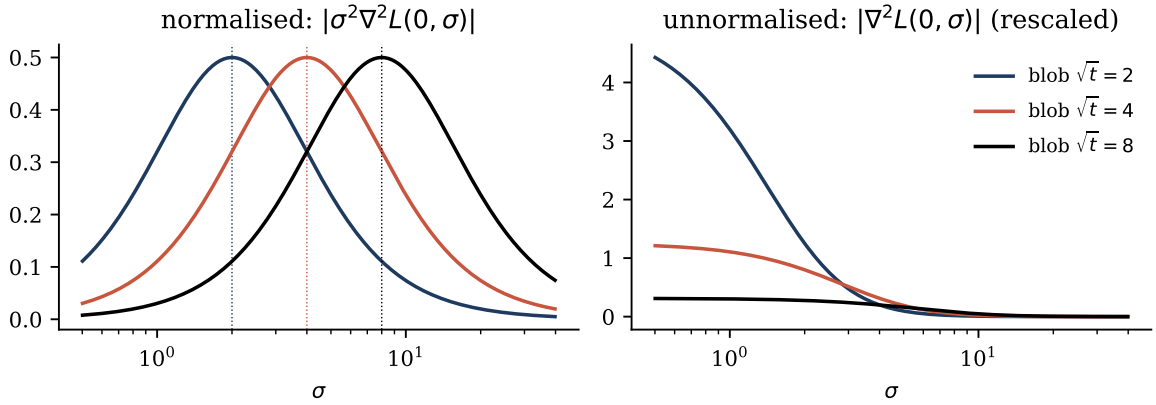


Figure 3: Left: normalised Laplacian response at the centre of three Gaussian blobs of different sizes, as a function of σ; each curve peaks exactly at the blob's own scale (dotted lines), with the same height. Right: without the factor σ^2 the responses simply decrease with σ and no scale is selected.

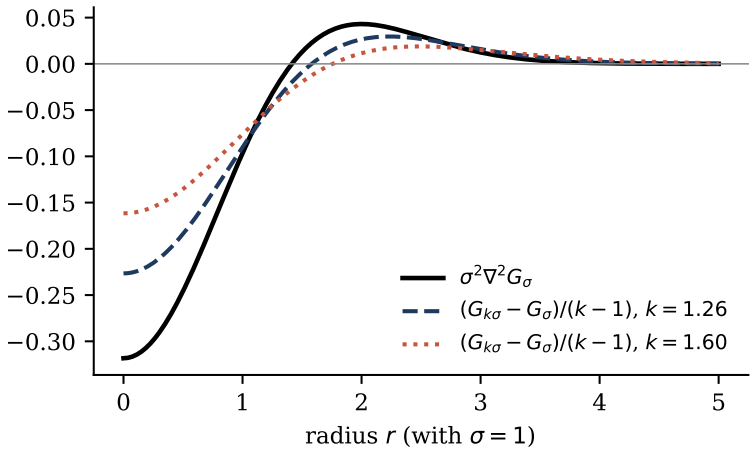


Figure 4: Radial profiles of the normalised LoG kernel and of the DoG kernel divided by $(k-1)$. The DoG has the same centre–surround shape; the discrepancy (essentially a change of amplitude and of effective scale, the same at every scale, so harmless for comparing extrema) vanishes as $k \to 1$.

4 SIFT: the Scale-Invariant Feature Transform

SIFT [1] consists of two independent parts:

1. a **detector**, which finds *frames* (x, y, s, θ): a position, a scale and an orientation, i.e. an oriented disc in the image;
2. a **descriptor**, which turns the image content inside each frame into a vector $\mathbf{d} \in \mathbb{R}^{128}$.

They are often used together, but each can be combined with other methods.

4.1 The detector

4.1.1 Building the DoG pyramid

The scale space is sampled in *octaves*; one octave corresponds to a doubling of σ. Each octave is divided into S intervals (Lowe and OpenCV use $S = 3$; OpenCV calls it `nOctaveLayers`), so consecutive scales differ by the factor $k = 2^{1/S}$.

1. The input image is (optionally) upsampled by a factor 2 and blurred to a base scale $\sigma_0 = 1.6$ (assuming the camera already introduced a blur of about 0.5 pixels).

2. Within an octave, $S+3$ Gaussian images $L(\cdot, \sigma_0 k^i)$, $i = 0, \ldots, S+2$, are computed incrementally using the semigroup property, and $S+2$ DoG images are obtained by subtracting neighbours.

3. The Gaussian image with twice the base scale is subsampled by a factor 2 and becomes the base of the next octave. Subsampling is legitimate because the image is already smooth enough at that scale, and it keeps the cost of each octave four times smaller than the previous one.

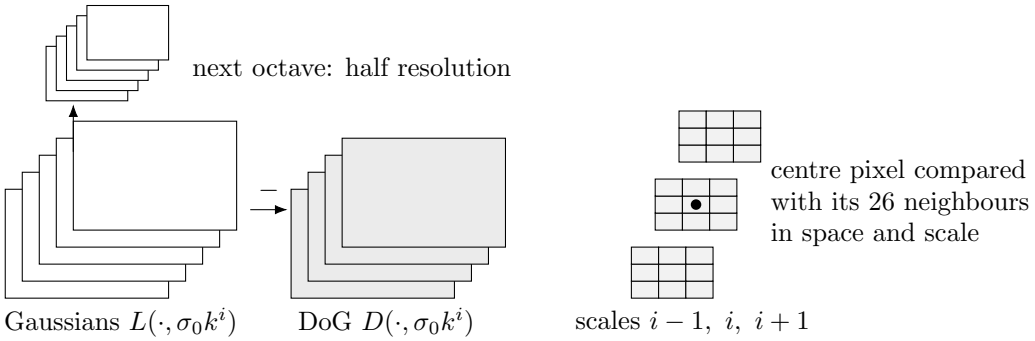


Figure 5: The DoG pyramid. Candidate keypoints are local extrema of D in a $3 \times 3 \times 3$ neighbourhood (8 neighbours at the same scale, 9 above and 9 below).

4.1.2 Extrema detection and sub-pixel refinement

A sample (x, y, σ) is a *candidate* if D there is strictly larger (or strictly smaller) than at all its 26 neighbours. Because the grid in (x, y, σ) is coarse, the true extremum lies somewhere between samples. Let $\mathbf{z} = (x, y, \sigma)^\top$ and expand D around the candidate $\mathbf{z}_0$ to second order, with $\boldsymbol{\delta} = \mathbf{z} - \mathbf{z}_0$:

$$D(\mathbf{z}_0 + \boldsymbol{\delta}) \approx D + \mathbf{g}^\top \boldsymbol{\delta} + \tfrac{1}{2} \boldsymbol{\delta}^\top \mathbf{H} \boldsymbol{\delta},$$

where $\mathbf{g}$ and $\mathbf{H}$ are the gradient and 3×3 Hessian of D at $\mathbf{z}_0$, estimated by finite differences. Setting the derivative to zero gives the offset and the value at the refined extremum:

$$\hat{\boldsymbol{\delta}} = -\mathbf{H}^{-1} \mathbf{g}, \qquad D(\hat{\mathbf{z}}) = D + \tfrac{1}{2}\, \mathbf{g}^\top \hat{\boldsymbol{\delta}}. \tag{4.1}$$

(The second formula follows by substituting $\hat{\boldsymbol{\delta}}$ in the expansion: $\mathbf{g}^\top \hat{\boldsymbol{\delta}} + \frac{1}{2} \hat{\boldsymbol{\delta}}^\top \mathbf{H} \hat{\boldsymbol{\delta}} = \mathbf{g}^\top \hat{\boldsymbol{\delta}} - \frac{1}{2} \mathbf{g}^\top \hat{\boldsymbol{\delta}}$.) If some component of $\hat{\boldsymbol{\delta}}$ exceeds 0.5, the extremum is closer to another sample and the procedure is repeated there. The final (x, y, σ) have sub-pixel and sub-scale accuracy.

4.1.3 The contrast (peak) threshold

Definition 4.1 (Contrast test)**.** A refined extremum is discarded when $|D(\hat{\mathbf{z}})| < \tau$, with intensities scaled to $[0, 1]$. Lowe uses $\tau = 0.03$.

Why is such a test meaningful? $|D|$ is (up to the constant $k-1$) the normalised Laplacian response, i.e. it measures *how much the blob stands out from its surround.* Image noise adds a random perturbation to D whose magnitude does not depend on the image content. When $|D|$ is of the order of the noise level, the very existence of the extremum, its location and its scale are determined by the noise rather than by the scene, so the same extremum will most likely not be found again in another picture of the scene. High-contrast extrema, on the contrary, are stable under noise, small resampling errors and moderate illumination changes. The threshold is therefore a trade-off between *quantity* (more features, better coverage of the object, more chances to match when it is partially occluded) and *quality* (stability and distinctiveness of each feature),

and it also controls the cost of all subsequent stages, which grows with the number of features (matching two images with n_1 and n_2 features costs $O(n_1 n_2)$, see Section 5.2).

Remark 4.2 (The parameter in the notebook). The helper `extract_sift(image, peak_thresh)` forwards `peak_thresh` to OpenCV as `contrastThreshold`. OpenCV divides it by `nOctaveLayers` (= 3) before applying the test, so OpenCV's `contrastThreshold`= 0.09 corresponds to Lowe's $\tau = 0.03$. The notebook's default `0.01` is therefore very permissive. The exact numerical scale differs from VLFeat's `PeakThresh` used in the original practical, but the meaning is the same: larger values keep fewer, higher-contrast features.

4.1.4 Edge responses

The DoG responds strongly along edges, not only at blobs. A point on an edge is well localised across the edge but poorly localised along it, so it is unstable. Let $\mathbf{H}_2 = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$ be the spatial Hessian, with eigenvalues $\alpha \geq \beta$ (the principal curvatures). On an edge one curvature is large and the other small.

Proposition 4.3 (Edge test). *If $\alpha = r\beta$ with α, β of the same sign and $r \geq 1$, then*

$$\frac{\operatorname{tr}(\mathbf{H}_2)^2}{\det(\mathbf{H}_2)} = \frac{(\alpha+\beta)^2}{\alpha\beta} = \frac{(r+1)^2}{r},$$

which is increasing in r. Hence requiring $\operatorname{tr}(\mathbf{H}_2)^2/\det(\mathbf{H}_2) < (r_0+1)^2/r_0$ (and $\det \mathbf{H}_2 > 0$) rejects points whose curvature ratio exceeds r_0, without computing eigenvalues.

Lowe and OpenCV use $r_0 = 10$ (OpenCV's `edgeThreshold`).

4.1.5 Orientation assignment

To achieve rotation covariance each keypoint receives a reference orientation computed from the image itself. In the Gaussian image $L(\cdot, \sigma)$ closest to the keypoint scale, gradient orientations of the pixels in a neighbourhood are accumulated into a 36-bin histogram (bins of 10°), each sample weighted by its gradient magnitude and by a Gaussian window of standard deviation 1.5σ centred at the keypoint. The highest peak gives the orientation θ (refined by fitting a parabola to the peak and its two neighbours).

Remark 4.4. Every other peak whose height is at least 80% of the highest one creates an *additional* keypoint at the same location and scale but with a different orientation. Thus two frames may share (x, y, s) and differ only by θ. Keep this in mind when you look at nearest neighbours in the descriptor space (Section 5.3).

4.2 Frames as similarity transformations

A detected frame $f = (x, y, s, \theta)$ is best thought of not as four numbers but as a *transformation*. Consider the *canonical frame*: the unit circle centred at the origin of an auxiliary plane with coordinates $\mathbf{u} = (u_1, u_2)^\top$, with reference direction along the positive u_1 axis. The frame f is the image of the canonical frame by the similarity

$$A_f(\mathbf{u}) = s\, R(\theta)\, \mathbf{u} + \begin{bmatrix} x \\ y \end{bmatrix}. \tag{4.2}$$

Indeed A_f maps the origin to the centre (x, y), the unit circle to the circle of radius s around it, and the reference direction $(1, 0)^\top$ to the direction $(\cos\theta, \sin\theta)^\top$. Conversely, every similarity (Section 6.1) of the form (4.2) with $s > 0$ defines a frame, so *frames and similarities are in one-to-one correspondence.*

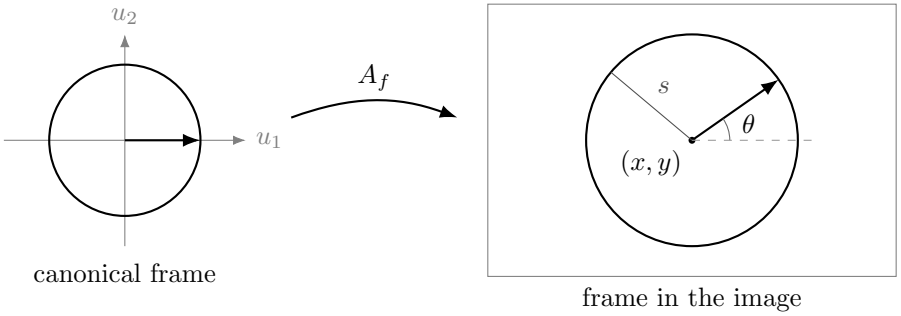


Figure 6: A SIFT frame is the image of the canonical oriented unit circle under a similarity A_f. (Drawn with y pointing up for readability; in image coordinates y points down.)

4.3 Covariance of the detector, invariance of the descriptor

Definition 4.5 (Covariant frame detector)**.** Let T be a similarity. A frame detector is *covariant to similarities* if, whenever f is detected in I, the frame f' with $A_{f'} = T \circ A_f$ is detected in $T \cdot I$.

In words: the frame detected in the transformed image is the transformed frame. The construction of Section 4.1 has this property (up to discretisation): positions are covariant because convolution commutes with translations, scales because of Proposition 3.2, and orientations because of Proposition 2.4(ii).

Proposition 4.6 (Normalisation yields invariance)**.** *Let the descriptor of frame f in image I be any function Ψ of the* normalised patch $I \circ A_f$ *(the image resampled in the canonical frame). If the detector is covariant, the descriptor is invariant to similarities.*

Proof. For the transformed image $T \cdot I = I \circ T^{-1}$ and the corresponding frame $A_{f'} = T \circ A_f$,

$$(T \cdot I) \circ A_{f'} = I \circ T^{-1} \circ T \circ A_f = I \circ A_f,$$

so the normalised patches, and hence the descriptors, coincide. □

Figure 7 shows the effect on a real image: the frames detected on the rotated and rescaled copy cover the same physical structures as the original ones.

4.4 Repeatability

Definition 4.7 (Repeatability [13])**.** Let I_1, I_2 be two images of a scene related by a *known* transformation T. A frame f of I_1 is *repeated* if I_2 contains a frame f' whose region overlaps $T(f)$ well (e.g. intersection over union above 0.6). The repeatability rate is the number of repeated frames divided by the smaller of the two numbers of frames detected in the common part of the images.

Covariance is a mathematical idealisation; actual repeatability is reduced by everything that is not a similarity of the image function: resampling and discretisation, sensor noise, illumination and exposure differences, perspective distortion (a plane seen from another viewpoint is related by a *homography*, not a similarity), non-planar 3D structure with self-occlusions, and objects that appear in only one of the images. A synthetically rotated and scaled copy is affected only by the first item, a second photograph by all of them.

4.5 The descriptor

The SIFT descriptor is a *histogram of gradient orientations*, computed in the normalised coordinate system of the frame.

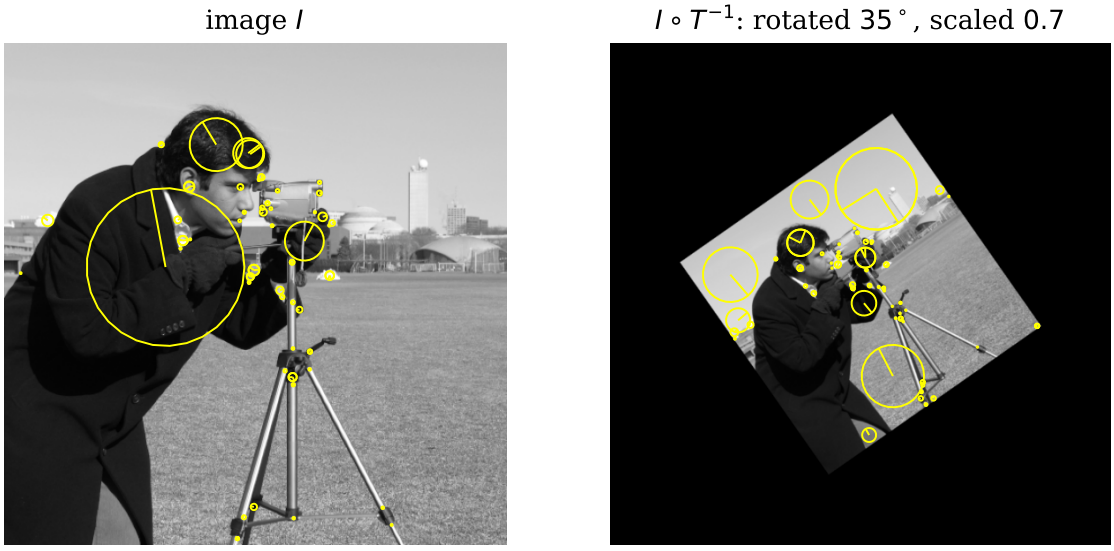


Figure 7: The 70 strongest SIFT frames of an image (left) and of a rotated and downscaled copy (right). Circles have radius s and the segment shows θ. Most frames reappear on the same structures with radii multiplied by 0.7 and orientations shifted by the rotation angle.

1. **Normalised window.** Around the keypoint a square window aligned with θ is divided into a 4×4 grid of spatial cells; each cell has side proportional to the scale ($3s$ in Lowe's and OpenCV's implementation). Sample positions are rotated by $-\theta$ and scaled by $1/s$, so that the grid lives in the canonical frame.
2. **Relative gradient orientations.** At each sample the gradient of $L(\cdot, \sigma)$ is computed and its orientation is expressed *relative to* θ, i.e. $\phi - \theta$.
3. **Weighting.** Each sample contributes its gradient magnitude multiplied by a Gaussian window centred at the keypoint (standard deviation equal to half the window width), which lowers the influence of the window borders, where misregistration matters most.
4. **Histograms.** In each of the 16 cells an 8-bin histogram of relative orientations (45° per bin) is accumulated: $4 \times 4 \times 8 = 128$ numbers. Each sample is spread over the 2×2 neighbouring cells and the 2 neighbouring orientation bins with weights $(1 - d)$, where d is the normalised distance to the bin centre (*trilinear interpolation*), so that the descriptor varies *continuously* when a sample moves across a bin boundary.
5. **Normalisation.** The vector is normalised to unit ℓ_2 norm, every entry is clipped at 0.2, and the vector is normalised again.

Remark 4.8 (OpenCV specifics). OpenCV multiplies the final unit vector by 512 and saturates to $[0, 255]$, so descriptor entries are floats in $[0, 255]$ with norm close to 512. A common positive factor does not change nearest neighbours, rankings of distances or ratios of distances, so it is irrelevant for matching.

4.6 Properties of the SIFT descriptor

Proposition 4.9 (Invariance to affine illumination changes)**.** *If the intensities inside the window change as $I \mapsto aI + b$ with $a > 0$, the SIFT descriptor (before clipping) is unchanged.*

Proof. By Proposition 2.4(i) every gradient is multiplied by a: orientations, hence bin assignments, are unchanged, and every histogram entry is multiplied by a. The ℓ_2 normalisation removes a. The offset b never appears since only derivatives are used. □

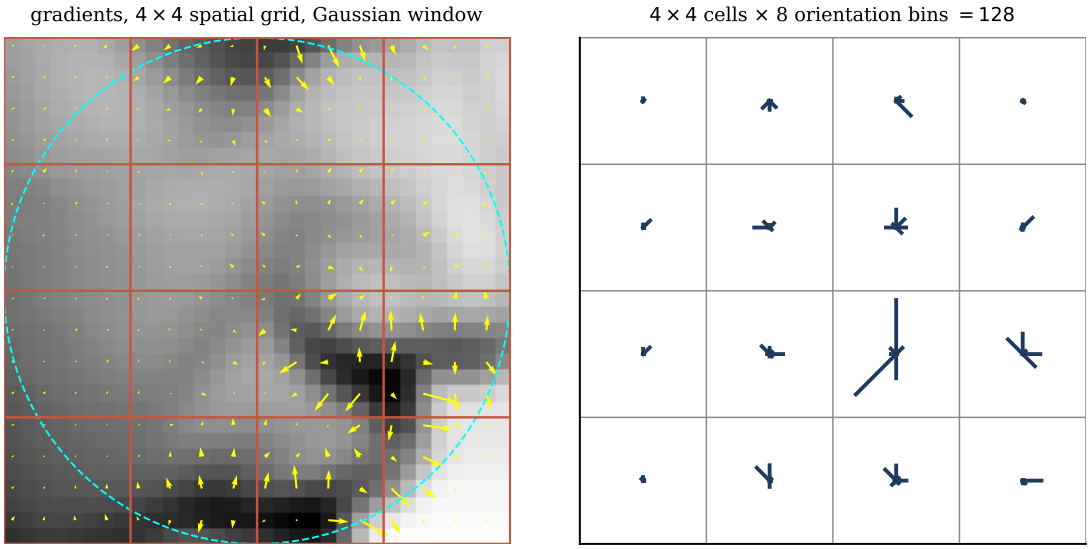


Figure 8: Left: gradients of a patch, the 4×4 spatial grid and the Gaussian weighting window. Right: the 16 orientation histograms (each segment's length is the weight of an orientation bin). Concatenating them gives the 128-dimensional descriptor.

Clipping at 0.2 adds robustness to *non-linear* photometric effects (saturation, specular highlights, illumination varying with 3D surface orientation), which tend to change the magnitude of a few large gradients much more than their orientations: after clipping, no single gradient can dominate the vector.

Further properties, each following from a step of the construction:

- **Similarity invariance**: by Proposition 4.6, because the descriptor is computed in the canonical frame and orientations are taken relative to θ.
- **Tolerance to small misalignments and small deformations**: within a cell, the histogram does not depend on *where* exactly each gradient lies (it is a sum over the cell), and trilinear interpolation makes the dependence on positions near cell borders continuous. A shift by a fraction of a cell, a slight perspective or affine distortion, or an error of a few degrees in θ only changes the descriptor slightly.
- **Distinctiveness**: the spatial layout is preserved at a coarse level (4×4 cells), so different structures give different descriptors.
- **Compactness**: 128 numbers, independent of the scale of the frame.

To appreciate the second item, it helps to quantify how a pixel-wise comparison behaves.

Proposition 4.10 (Sensitivity of pixel-wise comparison to shifts)**.** *Let P be a patch and $P_{\boldsymbol{\delta}}(\mathbf{x}) = P(\mathbf{x} + \boldsymbol{\delta})$ a copy shifted by a small $\boldsymbol{\delta}$. Then, to first order,*

$$\sum_{\mathbf{x}} \big(P_{\boldsymbol{\delta}}(\mathbf{x}) - P(\mathbf{x})\big)^2 \approx \sum_{\mathbf{x}} \big(\nabla P(\mathbf{x})^\top \boldsymbol{\delta}\big)^2 = \boldsymbol{\delta}^\top \Big(\sum_{\mathbf{x}} \nabla P \, \nabla P^\top\Big) \boldsymbol{\delta}.$$

Proof. Taylor expansion $P(\mathbf{x} + \boldsymbol{\delta}) \approx P(\mathbf{x}) + \nabla P(\mathbf{x})^\top \boldsymbol{\delta}$. □

The matrix in the middle is the *structure tensor* of the patch. For a textured patch (exactly the kind of patch a detector selects) it is large, so a sub-pixel misalignment already produces a large sum of squared differences. Likewise $\sum_{\mathbf{x}} (aP + b - P)^2$ grows with $|a - 1|$ and $|b|$: pixel values carry illumination along with structure.

Remark 4.11 (RootSIFT). Arandjelović and Zisserman [9] observed that comparing SIFT vectors with the Hellinger kernel instead of the Euclidean distance improves matching and retrieval. In

practice this amounts to ℓ_1-normalising each descriptor and taking the element-wise square root ("RootSIFT"); the same square-root trick reappears in Section 10.5.

5 Matching descriptors

5.1 Nearest-neighbour matching

Let $\{\mathbf{d}_i^1\}_{i=1}^{n_1}$ and $\{\mathbf{d}_j^2\}_{j=1}^{n_2}$ be the descriptors of two images. Since descriptors of the same physical point should be close, the simplest rule is:

Definition 5.1 (Nearest-neighbour (NN) matching)**.** Each feature i of image 1 is matched to

$$\nu(i) = \operatorname*{arg\,min}_{1 \le j \le n_2} \left\| \mathbf{d}_i^1 - \mathbf{d}_j^2 \right\| .$$

The output is the list of pairs $(i, \nu(i))$, which in the notebook is stored as an integer array with one row `[i, j]` per feature of image 1.

Properties worth keeping in mind:

- Every feature of image 1 receives a match, *even if its physical point is not visible in image* 2 (occluded, outside the field of view, or simply not detected there). Such features necessarily produce wrong matches. With clutter and partial overlap, these can be the majority.
- The map ν need not be injective: several features of image 1 can be matched to the same feature of image 2 (e.g. on repeated structures such as windows of a facade).
- Minimising the distance or the *squared* distance gives the same $\nu(i)$, since $t \mapsto t^2$ is increasing on $[0, \infty)$. This is no longer true for quantities that are not monotone transformations of the distance, such as ratios (see Section 5.3).

Definition 5.2 (Mutual nearest neighbours)**.** A pair (i, j) is a *mutual* NN match if $j = \nu(i)$ and i is also the nearest neighbour of j among the descriptors of image 1. This "cross-check" removes many-to-one matches.

5.2 Computing all distances efficiently

Proposition 5.3 (Expansion of the squared distance)**.** *For* $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$, $\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}^\top \mathbf{b}$. *Consequently, if the rows of* $X \in \mathbb{R}^{n_1 \times d}$ *and* $Y \in \mathbb{R}^{n_2 \times d}$ *are the descriptors, the matrix* $D_{ij} = \|\mathbf{x}_i - \mathbf{y}_j\|^2$ *is*

$$D = \mathbf{r}\,\mathbf{1}^\top + \mathbf{1}\,\mathbf{s}^\top - 2XY^\top, \qquad r_i = \|\mathbf{x}_i\|^2 ,\ s_j = \|\mathbf{y}_j\|^2 .$$

Proof. Expand $(\mathbf{a} - \mathbf{b})^\top (\mathbf{a} - \mathbf{b})$ and use $\mathbf{a}^\top \mathbf{b} = \mathbf{b}^\top \mathbf{a}$. □

This is how the notebook's helper `pairwise_squared_l2` works. Its advantages:

- the dominant cost is one matrix product $XY^\top$, which BLAS executes very efficiently;
- memory is $O(n_1 n_2)$ for D, instead of $O(n_1 n_2 d)$ for the naive broadcast `X[:, None, :] - Y[None, :, :]` (with $n_1 = n_2 = 5000$ and $d = 128$ in `float32` this naive tensor would take about 12.8 GB, the matrix D only 100 MB).

The total cost is $O(n_1 n_2 d)$ operations either way, quadratic in the number of features, which is one of the motivations for the contrast threshold and for the indexing structures of Section 10.3.

Pitfall (Cancellation). In floating point, $\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a}^\top \mathbf{b}$ subtracts large, nearly equal numbers when $\mathbf{a} \approx \mathbf{b}$; the result can come out slightly *negative*. That is why the helper clamps it with `np.maximum(D, 0)`. Take a square root only after clamping.

5.3 Lowe's ratio test

5.3.1 Motivation

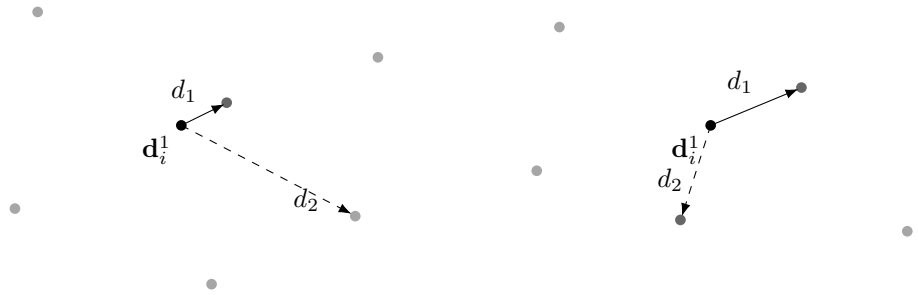
A small NN distance alone is a poor indicator of a correct match: some descriptors (low-texture, repetitive patterns) are close to many others, and the typical distance varies a lot across the descriptor space. What distinguishes a reliable match is that it is *much closer than any alternative*. Let, for feature i of image 1,

$$d_1(i) = \min_j \left\| \mathbf{d}_i^1 - \mathbf{d}_j^2 \right\|, \qquad d_2(i) = \text{the second smallest value of } \left\| \mathbf{d}_i^1 - \mathbf{d}_j^2 \right\|.$$

Definition 5.4 (Ratio test)**.** The match $(i, \nu(i))$ is accepted if

$$\rho(i) = \frac{d_1(i)}{d_2(i)} < \tau_\rho,$$

typically with $\tau_\rho = 0.8$ (the notebook's `NN_threshold`).



distinctive: $d_1 \ll d_2$, accepted ambiguous: $d_1 \approx d_2$, rejected

Figure 9: The ratio test in descriptor space (schematically in 2D). Grey dots are descriptors of image 2.

5.3.2 Why the second neighbour?

Think of d_2 as an estimate of *the distance at which wrong matches start to appear* around $\mathbf{d}_i^1$. If the point is visible in image 2, its correct match is at the first position, and the second neighbour is necessarily a wrong match; if the point is not visible, both are wrong matches. Wrong matches at the first and second positions come from the same "background" distribution and have similar distances, so $\rho \approx 1$; a correct match is closer than that background, so $\rho \ll 1$. The ratio is thus a *local, self-calibrated* measure of how much the best match stands out, much more informative than a global threshold on d_1. On his data Lowe reports that $\tau_\rho = 0.8$ removes about 90% of the false matches while discarding less than 5% of the correct ones [1].

Some properties and caveats:

- The ratio is *scale free*: multiplying all descriptors by a constant (as OpenCV does) does not change it.
- It is defined on *distances*. If you compute it from squared distances you obtain ρ^2, and the threshold 0.8 then has the meaning of $\sqrt{0.8} \approx 0.89$ on distances: a different, more permissive test. Be consistent with the definition you are asked to use.
- The denominator can vanish if image 2 contains two identical descriptors (or if both neighbours coincide with the query); your code must not divide by zero. Decide explicitly what ratio such a degenerate case should receive, keeping in mind that a vanishing d_2 means the match is *not* distinctive.

- Image 2 must contain at least two descriptors for d_2 to exist.
- Since SIFT may create several frames at the same location with different orientations (Section 4.1), the second neighbour is occasionally another descriptor of the same physical point; the test then rejects a correct match. This is a known, accepted loss.
- Smaller τ_ρ gives fewer but cleaner matches (higher precision, lower recall).

5.3.3 Finding the two smallest values

Sorting every row of the distance matrix costs $O(n_2 \log n_2)$ per row, but only the two smallest entries are needed, which can be done in $O(n_2)$ per row (a partial selection, see Appendix A). For many queries against a large set, a KD-tree queried with $k = 2$ neighbours (Section 10.3) is another option.

5.4 Evaluating a set of matches

If ground truth is available, a set of tentative matches is evaluated by its *precision* (fraction of the returned matches that are correct) and *recall* (fraction of the correct matches that are returned). Varying τ_ρ traces a precision–recall curve. Without ground truth, the geometric verification of Section 8 plays this role: it splits the tentative matches into inliers and outliers of a transformation.

6 Planar geometric transformations

Geometric verification and augmented reality require a model of how points of one image map to points of another. This section reviews the standard hierarchy of planar transformations [3, 14].

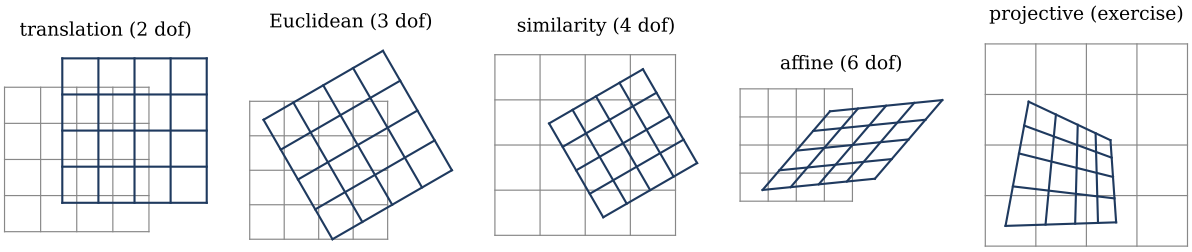


Figure 10: A square grid (grey) and its image (blue) under transformations of increasing generality. Each class contains the previous ones. Counting the degrees of freedom of the last one is left to you.

6.1 Euclidean, similarity and affine transformations

Definition 6.1. A map $T : \mathbb{R}^2 \to \mathbb{R}^2$ is

- a *translation* if $T(\mathbf{x}) = \mathbf{x} + \mathbf{t}$;
- a *Euclidean* (rigid) transformation if $T(\mathbf{x}) = R(\theta)\mathbf{x} + \mathbf{t}$;
- a *similarity* if $T(\mathbf{x}) = s\,R(\theta)\mathbf{x} + \mathbf{t}$ with $s > 0$;
- an *affine* transformation if $T(\mathbf{x}) = A\mathbf{x} + \mathbf{t}$ with $A \in \mathbb{R}^{2\times 2}$ invertible.

The similarities are exactly the transformations that preserve *angles* and *ratios of lengths*; they are the transformations under which a disc remains a disc, which is why they are the natural group for circular SIFT frames.

Proposition 6.2 (The similarity group)**.** *Write* $T_{(s,\theta,\mathbf{t})}(\mathbf{x}) = sR(\theta)\mathbf{x} + \mathbf{t}$. *Then:*

(i) $R(\alpha)R(\beta) = R(\alpha+\beta)$, $R(\alpha)^{-1} = R(\alpha)^\top = R(-\alpha)$, *and* R *commutes with scalar multiplication.*

(ii) Composition: $T_{(s_1,\theta_1,\mathbf{t}_1)} \circ T_{(s_2,\theta_2,\mathbf{t}_2)} = T_{(s_1 s_2,\ \theta_1+\theta_2,\ s_1 R(\theta_1)\mathbf{t}_2+\mathbf{t}_1)}$.

(iii) The identity is $T_{(1,0,\mathbf{0})}$ *and every similarity has an inverse which is again a similarity. Hence the similarities form a group.*

Proof. (i) is the trigonometric addition formula in matrix form. (ii) is a direct computation: $s_1R(\theta_1)\big(s_2R(\theta_2)\mathbf{x}+\mathbf{t}_2\big)+\mathbf{t}_1 = s_1s_2R(\theta_1+\theta_2)\mathbf{x} + s_1R(\theta_1)\mathbf{t}_2+\mathbf{t}_1$. (iii) Solve $\mathbf{y} = sR(\theta)\mathbf{x}+\mathbf{t}$ for $\mathbf{x}$. □

Exercise 6.3. Write the inverse in (iii) explicitly in the form $T_{(s',\theta',\mathbf{t}')}$, and verify your answer with (ii).

Remark 6.4 (Complex notation). Identifying $\mathbf{x} = (x,y)^\top$ with $z = x+iy \in \mathbb{C}$, a similarity is $z \mapsto az+b$ with $a = se^{i\theta} \in \mathbb{C}\setminus\{0\}$ and $b = t_x + it_y$: multiplication by a rotates by $\arg a$ and scales by $|a|$. This makes the group structure transparent (composition of $z \mapsto a_1z+b_1$ and $z \mapsto a_2z+b_2$ is $z \mapsto a_1a_2z + a_1b_2 + b_1$) and is a convenient way to double-check computations with similarities.

Remark 6.5 (Angles). Angles are defined modulo 2π. If you compute an angle as a difference of two orientations, the result may fall outside $(-\pi,\pi]$; since cos and sin are 2π-periodic this does not matter for $R(\theta)$, but it matters if you *compare* angles or average them.

6.2 Homogeneous coordinates and the projective plane

Definition 6.6 (Homogeneous coordinates)**.** A point $(x,y) \in \mathbb{R}^2$ is represented by any vector $\lambda(x,y,1)^\top$, $\lambda \neq 0$. Conversely, $(X,Y,W)^\top$ with $W \neq 0$ represents the point $(X/W,\ Y/W)$. Two non-zero vectors represent the same point iff they are proportional; we write $\mathbf{x} \sim \mathbf{x}'$. The set of equivalence classes of $\mathbb{R}^3\setminus\{\mathbf{0}\}$ is the *projective plane* $\mathbb{P}^2$. Vectors with $W=0$ are *points at infinity* (directions).

Proposition 6.7 (Lines)**.** *The line* $ax+by+c=0$ *is represented by* $\mathbf{l} = (a,b,c)^\top$ *(also up to scale), and a point* $\mathbf{x}$ *lies on it iff* $\mathbf{l}^\top\mathbf{x} = 0$. *The line through two points is* $\mathbf{l} = \mathbf{x}_1 \times \mathbf{x}_2$, *and the intersection of two lines is* $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$.

Proof. $\mathbf{x}_1 \times \mathbf{x}_2$ is orthogonal to both $\mathbf{x}_1$ and $\mathbf{x}_2$, i.e. $(\mathbf{x}_1\times\mathbf{x}_2)^\top\mathbf{x}_k = 0$ for $k=1,2$; the second statement is the dual one. □

The great benefit of homogeneous coordinates is that all the transformations above become *linear* maps of $\mathbb{R}^3$:

$$\underbrace{\begin{bmatrix} sR(\theta) & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{\text{similarity}}, \qquad \underbrace{\begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}}_{\text{affine}},$$

and composition becomes matrix multiplication.

Definition 6.8 (Homography)**.** A *homography* (projective transformation) of $\mathbb{P}^2$ is a map $\mathbf{x} \mapsto H\mathbf{x}$ where $H \in \mathbb{R}^{3\times3}$ is invertible. In inhomogeneous coordinates,

$$(x,y) \longmapsto \left(\frac{h_{11}x+h_{12}y+h_{13}}{h_{31}x+h_{32}y+h_{33}},\ \frac{h_{21}x+h_{22}y+h_{23}}{h_{31}x+h_{32}y+h_{33}}\right).$$

Proposition 6.9 (Basic properties of homographies)**.** *(i)* H *and* λH *(*$\lambda \neq 0$*) define the same map: a homography is determined by its matrix* up to scale.

(ii) Homographies map lines to lines: if $\mathbf{l}^\top \mathbf{x} = 0$ *then* $(H^{-\top}\mathbf{l})^\top (H\mathbf{x}) = 0$. *They therefore preserve collinearity and incidence, but not parallelism, lengths, angles or ratios of lengths in general.*

(iii) Affine transformations are exactly the homographies with last row proportional to $(0, 0, 1)$, *i.e. those that map the line at infinity to itself.*

(iv) The inverse map is given by H^{-1}, *and compositions by matrix products.*

Proof. (i) the scale cancels in the division. (ii) $(H^{-\top}\mathbf{l})^\top H\mathbf{x} = \mathbf{l}^\top H^{-1} H\mathbf{x} = \mathbf{l}^\top \mathbf{x} = 0$. (iii) a point at infinity $(X, Y, 0)^\top$ is mapped to a point at infinity for all X, Y iff $h_{31} = h_{32} = 0$. (iv) is clear. □

6.2.1 Why homographies appear in the assignment

Proposition 6.10 (Images of a plane are related by a homography)**.** *Let a pinhole camera project a 3D point* $\mathbf{X}$ *to* $\mathbf{x} \sim K[R \mid \mathbf{t}]\,\tilde{\mathbf{X}}$, *with* K *the (invertible) calibration matrix and* $\tilde{\mathbf{X}} = (X, Y, Z, 1)^\top$. *If the scene points lie on the plane* $Z = 0$, *then* $\mathbf{x} \sim H(X, Y, 1)^\top$ *with* $H = K[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$, *where* $\mathbf{r}_1, \mathbf{r}_2$ *are the first two columns of* R. *Consequently two images of the same plane are related by the homography* $H_2 H_1^{-1}$.

Proof. With $Z = 0$ the third column of R is multiplied by zero, which gives the formula for H. When the plane does not pass through the camera centre, H is invertible, and eliminating $(X, Y, 1)^\top$ between the two views gives $\mathbf{x}_2 \sim H_2 H_1^{-1} \mathbf{x}_1$. □

A poster, a painting or a book cover is planar; a template image is (up to scaling) a fronto-parallel picture of it. Hence the map between the template and its appearance in a photograph is a homography. Another classical case: two pictures taken from the same centre with a rotating camera are related by $H = KRK^{-1}$ whatever the scene (this is the basis of panorama stitching).

Pitfall (Direction of the mapping). A homography estimated from correspondences $(\mathbf{x}_1^i, \mathbf{x}_2^i)$ with $\mathbf{x}_2^i \sim H\mathbf{x}_1^i$ maps image 1 *to* image 2. In the notebook, image 1 is the scene and image 2 the template. Check which direction your warping procedure needs (Section 9) before using H or H^{-1}.

6.3 The cross product as a linear map

Definition 6.11. For $\mathbf{a} = (a_1, a_2, a_3)^\top$, the matrix

$$[\mathbf{a}]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}$$

satisfies $\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_\times \mathbf{b}$ for all $\mathbf{b} \in \mathbb{R}^3$.

Proposition 6.12. *Let* $\mathbf{a} \neq \mathbf{0}$.

(i) $[\mathbf{a}]_\times^\top = -[\mathbf{a}]_\times$.

(ii) $\ker[\mathbf{a}]_\times = \operatorname{span}(\mathbf{a})$, *hence* $\operatorname{rank}[\mathbf{a}]_\times = 2$.

(iii) $\mathbf{a}^\top [\mathbf{a}]_\times = \mathbf{0}^\top$: *the three rows of* $[\mathbf{a}]_\times$ *are linearly dependent, with coefficients* a_1, a_2, a_3.

(iv) For non-zero $\mathbf{b}$: $\mathbf{a} \times \mathbf{b} = \mathbf{0} \iff \mathbf{b} = \lambda \mathbf{a}$ *for some* $\lambda \neq 0$.

Proof. (i) by inspection. (ii) $\mathbf{a} \times \mathbf{b} = \mathbf{0}$ iff $\mathbf{a}, \mathbf{b}$ are parallel, so the kernel is the line spanned by $\mathbf{a}$; by rank–nullity the rank is $3 - 1 = 2$. (iii) is (i) and (ii) combined: $\mathbf{a}^\top [\mathbf{a}]_\times = -([\mathbf{a}]_\times \mathbf{a})^\top = \mathbf{0}^\top$. (iv) restates (ii). □

Corollary 6.13 (Eliminating the unknown scale)**.** *For non-zero vectors,* $\mathbf{x}' \sim H\mathbf{x}$ *("there exists* $\lambda \neq 0$ *with* $\lambda\mathbf{x}' = H\mathbf{x}$*") is equivalent to the equation* $\mathbf{x}' \times (H\mathbf{x}) = \mathbf{0}$*, which no longer contains* λ *and is* linear in the entries of H*.*

This is the key trick of the next section. Note that by Proposition 6.12(ii)–(iii) the three scalar equations contained in $\mathbf{x}' \times (H\mathbf{x}) = \mathbf{0}$ are not independent.

6.4 Degrees of freedom and minimal samples

Definition 6.14. The *number of degrees of freedom* (dof) of a class of transformations is the number of independent real parameters needed to specify one of its elements (formally, the dimension of the group as a manifold).

Translations have 2 dof (t_x, t_y), Euclidean transformations 3, similarities 4 (s, θ, t_x, t_y), affine transformations 6 (four entries of A and two of $\mathbf{t}$).

A point correspondence $\mathbf{x} \leftrightarrow \mathbf{x}'$ imposes *two* scalar equations (one per coordinate). A *minimal sample* is a smallest set of correspondences whose equations determine the transformation, for points in *general position*. Configurations for which the equations become dependent, so that the solution is not unique, are called *degenerate*.

Example 6.15 (Affine transformations)**.** Write $T(\mathbf{x}) = A\mathbf{x} + \mathbf{t}$ and collect the unknowns in $\mathbf{p} = (a_{11}, a_{12}, t_x, a_{21}, a_{22}, t_y)^\top$. Each correspondence $(x, y) \mapsto (x', y')$ gives two linear equations,

$$\begin{bmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{bmatrix} \mathbf{p} = \begin{bmatrix} x' \\ y' \end{bmatrix}.$$

Six unknowns call for three correspondences. Stacking them, the 6×6 matrix is block diagonal (after reordering rows) with two copies of $M = \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{bmatrix}$, so its determinant is $\det(M)^2$. Now $\det M = 0$ iff the three points are *collinear* (it is twice the signed area of the triangle). Hence: *three non-collinear correspondences determine a unique affine map, and three collinear points are degenerate.*

Example 6.16 (Similarities)**.** Four dof: two point correspondences with distinct points suffice. A SIFT frame, however, carries four numbers (x, y, s, θ), so a single *frame* correspondence also provides four scalar constraints. This is why the RANSAC of Part 1 can sample one match at a time.

Remark 6.17 (Counting for matrices defined up to scale). When a transformation is represented by a matrix that is only defined up to a non-zero factor, the entries are not independent parameters: the overall scale must be removed from the count (or fixed by a normalisation). The worked example of Section 7.3 shows this for the projective line.

7 Homogeneous linear systems and the SVD

7.1 The problem

Many estimation problems in geometry reduce to a *homogeneous* linear system

$$A\mathbf{h} = \mathbf{0}, \qquad A \in \mathbb{R}^{m \times n},\ \mathbf{h} \in \mathbb{R}^n,$$

in which $\mathbf{h}$ is only defined up to scale (for instance the entries of a homography, or of a line $\mathbf{l}$). The trivial solution $\mathbf{h} = \mathbf{0}$ is useless, so we fix the scale by the constraint $\|\mathbf{h}\| = 1$. With noisy data there is generally no exact non-zero solution, and we solve instead

$$\min_{\mathbf{h} \in \mathbb{R}^n} \|A\mathbf{h}\|^2 \qquad \text{subject to} \qquad \|\mathbf{h}\| = 1. \tag{7.1}$$

7.2 Solution by the singular value decomposition

Theorem 7.1 (SVD)**.** *Every $A \in \mathbb{R}^{m\times n}$ factorises as $A = U\Sigma V^\top$, with $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ orthogonal and $\Sigma \in \mathbb{R}^{m\times n}$ zero except for the diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ (the singular values). The columns $\mathbf{v}_1, \ldots, \mathbf{v}_n$ of V are the right singular vectors; they are eigenvectors of $A^\top A$ with eigenvalues σ_i^2 (setting $\sigma_i = 0$ for $i > m$ when $m < n$).*

Theorem 7.2 (Homogeneous least squares)**.** *A minimiser of* (7.1) *is the right singular vector $\mathbf{v}_n$ associated with the smallest singular value σ_n, and the minimum value is σ_n^2. The minimiser is unique up to sign if $\sigma_{n-1} > \sigma_n$.*

Proof. Since U is orthogonal, $\|A\mathbf{h}\| = \left\|U\Sigma V^\top \mathbf{h}\right\| = \left\|\Sigma V^\top \mathbf{h}\right\|$. Put $\mathbf{y} = V^\top \mathbf{h}$; as V is orthogonal, $\|\mathbf{y}\| = \|\mathbf{h}\| = 1$, and $\mathbf{y}$ ranges over the whole unit sphere when $\mathbf{h}$ does. Then

$$\|A\mathbf{h}\|^2 = \sum_{i=1}^{n} \sigma_i^2 y_i^2 \;\geq\; \sigma_n^2 \sum_{i=1}^{n} y_i^2 = \sigma_n^2,$$

with equality for $\mathbf{y} = \mathbf{e}_n$, i.e. $\mathbf{h} = V\mathbf{e}_n = \mathbf{v}_n$. If $\sigma_{n-1} > \sigma_n$, equality forces $y_i = 0$ for $i < n$, so $\mathbf{y} = \pm\mathbf{e}_n$. □

Remark 7.3. Three practical observations.

- In the noise-free case, if the system has a one-dimensional solution space, then $\sigma_n = 0$ and $\mathbf{v}_n$ spans the null space of A.
- Equivalently, $\mathbf{v}_n$ is the eigenvector of $A^\top A$ with the smallest eigenvalue. Using the SVD of A directly is numerically preferable because forming $A^\top A$ squares the condition number.
- In NumPy, `U, S, Vt = np.linalg.svd(A)` returns $V^\top$; the right singular vectors are the *rows* of `Vt`, ordered by decreasing singular value, so $\mathbf{v}_n$ is `Vt[-1]`. With the default `full_matrices=True`, `Vt` is $n \times n$ even when $m < n$, so `Vt[-1]` is still a null vector in an under-determined minimal case.

Example 7.4 (Total least-squares line fitting)**.** Fitting a line $\mathbf{l} = (a, b, c)^\top$ to points (x_i, y_i) means solving $ax_i + by_i + c = 0$ for all i, i.e. $A\mathbf{l} = \mathbf{0}$ with rows $(x_i, y_i, 1)$. The following snippet solves it by Theorem 7.2.

```
import numpy as np

def fit_line_homogeneous(points):
    """Fit a line a*x + b*y + c = 0 to an (N, 2) array of points, N >= 2."""
    A = np.column_stack([points[:, 0], points[:, 1], np.ones(len(points))])
    _, S, Vt = np.linalg.svd(A)
    line = Vt[-1]                   # singular vector of smallest sigma
    return line / np.hypot(line[0], line[1])   # |a x + b y + c| = distance

rng = np.random.default_rng(0)
x = rng.uniform(0, 10, 50)
pts = np.column_stack([x, 2.0 * x - 1.0 + rng.normal(0, 0.05, 50)])
print(fit_line_homogeneous(pts))   # approx. +-(0.894, -0.447, -0.447)
```

7.3 The Direct Linear Transform

The *Direct Linear Transform* (DLT) is the general recipe for estimating a transformation that is linear in homogeneous coordinates:

1. write each correspondence as $\mathbf{x}_i' \sim M\mathbf{x}_i$;

2. eliminate the unknown scale with a "cross product" condition (Corollary 6.13), which is *linear* in the entries of M;
3. rewrite it as $A_i\,\mathbf{m} = \mathbf{0}$, where $\mathbf{m}$ is the vector of entries of M in a fixed order and A_i contains only known coordinates; keep only linearly independent rows;
4. stack the A_i into A and solve (7.1) by the SVD;
5. reshape $\mathbf{m}$ into M (with the same ordering) and, if desired, fix the scale.

Step 3 is where the work is: you must expand the products and collect, for each scalar equation, the coefficient of each unknown. We illustrate it on a *simpler* problem than the one of the assignment.

Example 7.5 (DLT on the projective line)**.** Points of a line are written in homogeneous coordinates as $\mathbf{x} = (x, 1)^\top \in \mathbb{R}^2$, and a 1D homography is $\mathbf{x}' \sim H\mathbf{x}$ with $H = \left[\begin{smallmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{smallmatrix}\right]$ invertible, i.e. $x' = (h_{11}x + h_{12})/(h_{21}x + h_{22})$.

Eliminating the scale. For non-zero $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$, $\mathbf{u} \sim \mathbf{v}$ iff $\det[\mathbf{u}\ \mathbf{v}] = u_1v_2 - u_2v_1 = 0$ (the 2D analogue of the cross product). With $\mathbf{u} = \mathbf{x}' = (x', 1)^\top$ and $\mathbf{v} = H\mathbf{x} = (h_{11}x + h_{12},\ h_{21}x + h_{22})^\top$:

$$x'(h_{21}x + h_{22}) - (h_{11}x + h_{12}) = 0.$$

Collecting coefficients. With $\mathbf{h} = (h_{11}, h_{12}, h_{21}, h_{22})^\top$ this reads

$$\underbrace{\begin{bmatrix} -x & -1 & x'x & x' \end{bmatrix}}_{A_i \in \mathbb{R}^{1\times 4}} \mathbf{h} = 0.$$

Counting. H has 4 entries but is defined up to scale, so it has $4 - 1 = 3$ dof. Each correspondence gives one equation; three correspondences are needed. If two of the x_i coincide, their rows are identical and the sample is degenerate.

Solving. With $n \geq 3$ correspondences, stack the rows into $A \in \mathbb{R}^{n\times 4}$ and take $\mathbf{h}$ as the last right singular vector.

The 2D case of the assignment follows exactly the same steps, with 3-vectors, the cross product of Section 6.3, and a 3×3 matrix flattened row by row as in the notebook.

Pitfall (Ordering). The order in which you list the unknowns in $\mathbf{h}$ when you build A_i must be the order that the final `reshape(3, 3)` assumes. NumPy reshapes in row-major ("C") order: the first three entries become the first row.

7.4 Algebraic error and data normalisation

The quantity $\|A\mathbf{h}\|$ minimised by the DLT is an *algebraic* error: it has no direct geometric meaning (it is not a distance in pixels). It works well when the data are well conditioned, but raw pixel coordinates are not: in a row of A for a homography, some entries are products of two coordinates ($\sim 10^5$ for a 500-pixel image), some are coordinates ($\sim 10^2$) and some are 1. Such disparities make the smallest singular vector sensitive to noise. Hartley [4] showed that a simple normalisation fixes this.

Definition 7.6 (Normalising similarity)**.** Given points $\mathbf{x}_1, \ldots, \mathbf{x}_N$ with centroid $\bar{\mathbf{x}}$ and mean distance to the centroid $\bar{r}$, let

$$T = \begin{bmatrix} \gamma & 0 & -\gamma\bar{x} \\ 0 & \gamma & -\gamma\bar{y} \\ 0 & 0 & 1 \end{bmatrix}, \qquad \gamma = \frac{\sqrt{2}}{\bar{r}}.$$

The normalised points $\hat{\mathbf{x}}_i = T\mathbf{x}_i$ have centroid $\mathbf{0}$ and mean distance $\sqrt{2}$ to it.

Proposition 7.7 (De-normalisation)**.** *Let T_1, T_2 normalise the points of images* 1 *and* 2*, and let $\hat{H}$ be estimated from the normalised correspondences $\hat{\mathbf{x}}_2^i \sim \hat{H}\hat{\mathbf{x}}_1^i$. Then $H = T_2^{-1}\hat{H}T_1$ satisfies $\mathbf{x}_2^i \sim H\mathbf{x}_1^i$.*

Proof. $\hat{\mathbf{x}}_2 \sim \hat{H}\hat{\mathbf{x}}_1$ means $T_2\mathbf{x}_2 \sim \hat{H}T_1\mathbf{x}_1$; multiply on the left by T_2^{-1}. □

```
def normalising_transform(points):
    """3x3 T such that the points T @ [x, y, 1] have zero mean, mean norm sqrt(2).
    """
    centroid = points.mean(axis=0)
    mean_dist = np.linalg.norm(points - centroid, axis=1).mean()
    gamma = np.sqrt(2.0) / max(mean_dist, 1e-12)
    return np.array([[gamma, 0.0, -gamma * centroid[0]],
                     [0.0, gamma, -gamma * centroid[1]],
                     [0.0, 0.0, 1.0]])
```

Normalisation is not required by the assignment, but it is cheap and is the standard way to obtain accurate homographies. Finally, the notebook fixes the arbitrary scale of the result by dividing by h_{33}; this is legitimate as long as $h_{33} \neq 0$, i.e. as long as the origin of image 1 is not sent to infinity, which is the case in any reasonable configuration.

8 Robust estimation with RANSAC

8.1 Why least squares is not enough

After the ratio test a sizeable fraction of tentative matches is still wrong. Least-squares estimators are extremely sensitive to such *outliers*: a single correspondence placed far enough can move the least-squares solution arbitrarily far (the estimator has a *breakdown point* of 0%). We need a method that first identifies which data agree with a common model, and only then fits the model to them.

8.2 The algorithm

RANSAC (RANdom SAmple Consensus) [5] is based on two observations: a model can be computed from a *minimal sample* (Section 6.4), and if that sample contains only inliers, the resulting model is approximately correct and is therefore supported by *all* the inliers.

Algorithm 1 RANSAC

Require: data $\{z_k\}_{k=1}^n$, minimal sample size m, residual function $r(\mathcal{M}, z)$, threshold ϵ, number of iterations N

$\mathcal{I}^\star \leftarrow \emptyset$, $\mathcal{M}^\star \leftarrow$ none
for it $= 1, \ldots, N$ **do**
  draw a random subset S of m data
  compute the model $\mathcal{M}_S$ from S ▷ exact fit, e.g. closed form
  if S is degenerate (no unique model) **then continue**
  $\mathcal{I}_S \leftarrow \{k : r(\mathcal{M}_S, z_k) < \epsilon\}$ ▷ consensus set
  if $|\mathcal{I}_S| > |\mathcal{I}^\star|$ **then** $\mathcal{I}^\star \leftarrow \mathcal{I}_S$, $\mathcal{M}^\star \leftarrow \mathcal{M}_S$
(optional) re-estimate $\mathcal{M}^\star$ by least squares on $\mathcal{I}^\star$, recompute $\mathcal{I}^\star$, repeat
return $\mathcal{M}^\star$, $\mathcal{I}^\star$

For geometric verification of matches:

- a datum z_k is a tentative match $(\mathbf{x}_1^k, \mathbf{x}_2^k)$ (possibly with full frames);

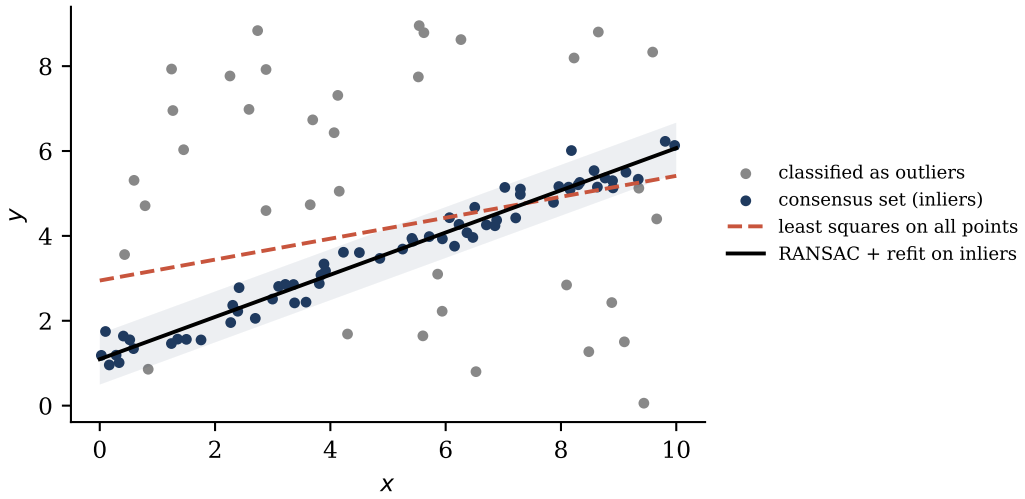


Figure 11: Line fitting with 40% outliers. Least squares on all points (dashed) is pulled away by the outliers; RANSAC (minimal sample: two points) finds the consensus set within the band of width 2ϵ, and a final least-squares fit on the inliers gives the solid line.

- the model is the transformation T (similarity, affine, homography, ...);
- the residual is the *transfer error* $r(T, z_k) = \left\| T(\mathbf{x}_1^k) - \mathbf{x}_2^k \right\|$, measured in pixels in image 2, and ϵ is a distance in pixels (the notebook's `dist_thresh`).

8.3 How many iterations?

Proposition 8.1. *Let w be the fraction of inliers and assume samples of m data are drawn independently and uniformly (ignoring the dependence created by sampling without replacement). The probability that at least one of N samples consists only of inliers is $1 - (1 - w^m)^N$. To make it at least p one needs*

$$N \;\geq\; \frac{\log(1-p)}{\log(1-w^m)}.$$

Proof. A sample is all-inlier with probability w^m; all N samples fail independently with probability $(1 - w^m)^N$. Require $(1 - w^m)^N \leq 1 - p$ and take logarithms (both are negative, which flips the inequality). □

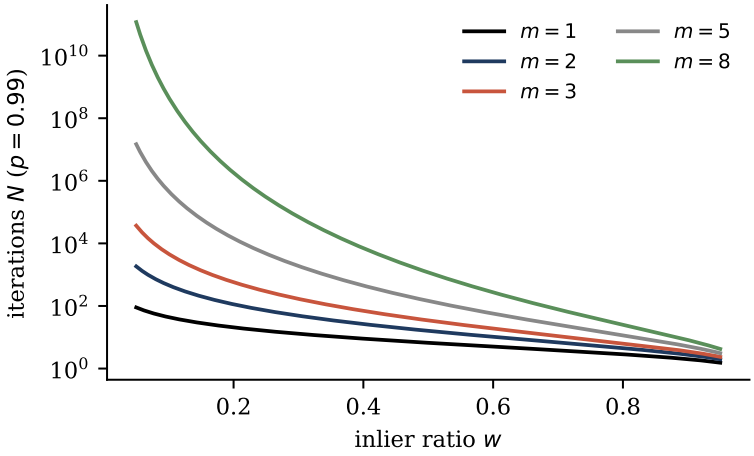


Figure 12: Number of iterations required for $p = 0.99$ as a function of the inlier ratio, for several minimal sample sizes m (logarithmic scale). The cost explodes with m when w is small.

Figure 12 shows the practical message: **the smaller the minimal sample, the cheaper RANSAC is.** With $w = 0.2$ and $p = 0.99$, one needs about 21 iterations if $m = 1$, 113 if $m = 2$, and about 14 000 if $m = 5$. This is the reason for using the full frame information (position, scale and orientation) to hypothesise a similarity from a *single* match. When $m = 1$ and the number of matches n is moderate, one can even test *every* match as a hypothesis (n iterations, cost $O(n^2)$), which makes the procedure deterministic.

Since w is unknown, a common strategy is *adaptive* RANSAC: start with a pessimistic w, and every time a larger consensus set is found, update $w = |\mathcal{I}^\star| / n$ and recompute N.

8.4 Choosing the threshold

If inliers are corrupted by isotropic Gaussian noise of standard deviation σ in each coordinate, the squared transfer error of an inlier follows (approximately) $\sigma^2 \chi^2_2$, and $\epsilon^2 = 5.99\,\sigma^2$ accepts 95% of the inliers. In practice ϵ is chosen empirically. A threshold that is too small rejects true inliers (especially if the model is only approximate, e.g. a similarity for a perspective view); a threshold that is too large accepts outliers and makes wrong models look good. Note that the transfer error is measured in image 2, so the appropriate value depends on the image resolution.

8.5 Refinement and variants

- **Re-estimation.** The model from the minimal sample is noisy. Re-fitting it by least squares on the whole consensus set (and possibly iterating "fit, recompute inliers") improves it significantly; this is the idea of LO-RANSAC. One can also re-fit a *richer* model (an affine map or a homography) to the inliers of a similarity, as suggested in the notebook.
- **Scoring.** Counting inliers treats all of them equally. MSAC scores a model by $\sum_k \min(r_k^2, \epsilon^2)$ instead, which also rewards precise models.
- **Degeneracy.** A minimal sample may not determine a model (two identical points for a similarity, collinear points for an affine map or a homography); such samples must be detected and skipped.
- **Many-to-one matches.** Several matches sharing the same feature can all be counted as inliers of the same model; enforcing one-to-one matches before RANSAC avoids inflating scores.
- **Randomness.** Results vary between runs. Fix the random seed if you need reproducible figures.

8.6 A generic implementation

The following implementation fits a line, i.e. a model different from the one of the assignment, but it shows the structure: a minimal solver, a residual, a consensus set and a final re-fit.

```
import numpy as np

def line_through(p, q):
    """Homogeneous line through two 2D points."""
    return np.cross(np.append(p, 1.0), np.append(q, 1.0))

def point_line_distance(line, points):
    a, b, c = line
    return np.abs(a * points[:, 0] + b * points[:, 1] + c) / np.hypot(a, b)

def ransac_line(points, n_iters=200, eps=0.5, seed=0):
    rng = np.random.default_rng(seed)
    best_inliers = np.empty(0, dtype=int)
```

```
    for _ in range(n_iters):
        i, j = rng.choice(len(points), size=2, replace=False)   # minimal sample
        line = line_through(points[i], points[j])
        if np.hypot(line[0], line[1]) < 1e-12:   # degenerate sample
            continue
        inliers = np.flatnonzero(point_line_distance(line, points) < eps)
        if len(inliers) > len(best_inliers):
            best_inliers = inliers
    # final least-squares re-fit on the consensus set
    best_line = fit_line_homogeneous(points[best_inliers])
    return best_line, best_inliers
```

9 Image warping, interpolation and compositing

Given a transformation between a template and a scene, augmented reality "pastes" a new image (of the same size as the template) onto the scene, deformed so that it occupies the place of the template. This requires three ingredients: a warping strategy, an interpolation scheme, and a compositing rule.

9.1 Forward versus inverse warping

Let T map source coordinates (template) to destination coordinates (scene).

- **Forward warping** visits every *source* pixel $\mathbf{q}$ and writes its colour at $T(\mathbf{q})$. Since $T(\mathbf{q})$ is generally not an integer position, the value must be rounded or "splatted"; worse, when T enlarges the image some destination pixels receive no value (*holes*), and when it shrinks it several source pixels compete for the same destination pixel.
- **Inverse warping** visits every *destination* pixel $\mathbf{p}$, computes the corresponding source location $\mathbf{c} = T^{-1}(\mathbf{p})$, and reads the source image there. Every destination pixel receives exactly one value, and there are no holes. The price is that $\mathbf{c}$ is not an integer position, so the source image must be *interpolated*.

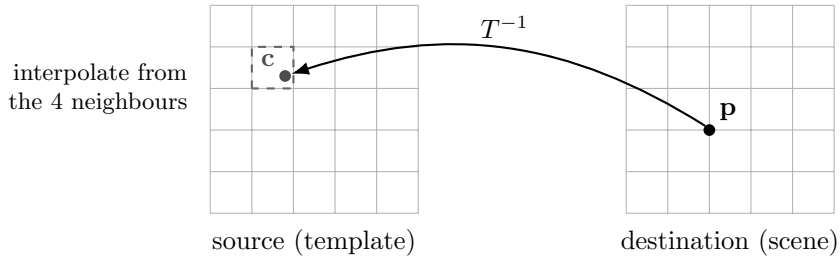


Figure 13: Inverse warping: for each destination pixel $\mathbf{p}$ (grid nodes are pixel positions) the source location $\mathbf{c} = T^{-1}(\mathbf{p})$ is computed and the source image is interpolated there from the four surrounding pixels.

With homographies in homogeneous coordinates, computing $\mathbf{c}$ means: form $(p_x, p_y, 1)^\top$, multiply by the appropriate 3×3 matrix, and divide by the third coordinate.

Pitfall. Inverse warping needs the map *from the destination to the source*. Check whether the matrix you have maps scene to template or template to scene (Section 6); in the latter case you need its inverse.

9.2 Interpolation

We want to evaluate a sampled image $f(n)$, known only at integer positions, at a real position.

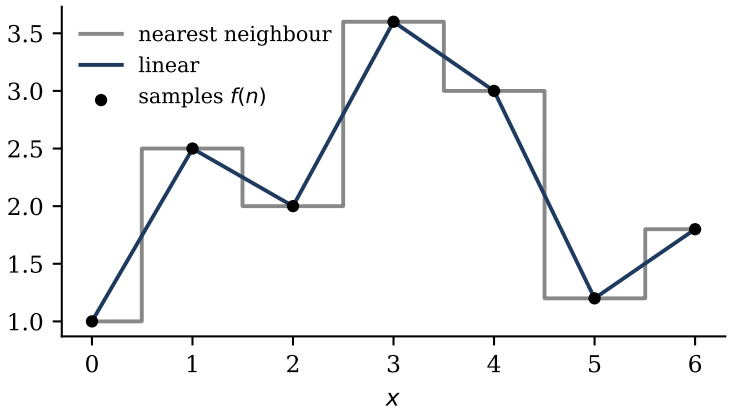


Figure 14: Nearest-neighbour interpolation produces a piecewise constant, discontinuous signal ("blocky" results in 2D); linear interpolation a continuous piecewise linear one.

Definition 9.1 (Linear interpolation in 1D)**.** For $c \in [n, n+1]$ with $n = \lfloor c \rfloor$ and $\phi = c - n \in [0, 1)$,

$$\hat{f}(c) = (1 - \phi)\, f(n) + \phi\, f(n+1).$$

Definition 9.2 (Bilinear interpolation)**.** Let I be an image with values at integer positions, and $\mathbf{c} = (c_x, c_y)$ a real position. Put $x_0 = \lfloor c_x \rfloor$, $y_0 = \lfloor c_y \rfloor$, $f_x = c_x - x_0$, $f_y = c_y - y_0$, and denote the four neighbours by

$$\mathrm{TL} = I(x_0, y_0), \quad \mathrm{TR} = I(x_0 + 1, y_0), \quad \mathrm{BL} = I(x_0, y_0 + 1), \quad \mathrm{BR} = I(x_0 + 1, y_0 + 1)$$

("top" is the smaller y, since y grows downwards). The bilinear interpolant is

$$\hat{I}(\mathbf{c}) = (1 - f_x)(1 - f_y)\,\mathrm{TL} + f_x(1 - f_y)\,\mathrm{TR} + (1 - f_x) f_y\,\mathrm{BL} + f_x f_y\,\mathrm{BR}.$$

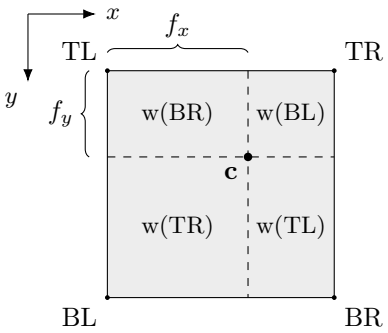


Figure 15: Bilinear interpolation in a unit cell (drawn with y pointing *down*, as in images). The weight of each corner is the area of the sub-rectangle *opposite* to it (the label w(TL) marks the area that gives the weight of TL, and so on): the closer the point to a corner, the larger that corner's weight.

Proposition 9.3 (Properties of bilinear interpolation)**.** *(i) It is linear interpolation along x on the two rows y_0 and $y_0 + 1$, followed by linear interpolation along y of the two results (or the other way round: the result is the same).*

(ii) The four weights are non-negative and sum to 1; hence $\hat{I}(\mathbf{c})$ is a convex combination of the four neighbours and lies between their minimum and maximum (for every colour channel).

(iii) $\hat{I}$ coincides with I at integer positions and is continuous across cell boundaries.

(iv) On each cell, $\hat{I}$ is the unique function of the form $a + bx + cy + dxy$ that takes the four corner values. It is linear along horizontal and vertical lines but not along diagonals.

Proof. (i) Interpolating along x gives $u_0 = (1 - f_x)\mathrm{TL} + f_x\mathrm{TR}$ and $u_1 = (1 - f_x)\mathrm{BL} + f_x\mathrm{BR}$; then $(1 - f_y)u_0 + f_y u_1$ expands to the formula. Starting with y gives the same four products. (ii) $f_x, f_y \in [0, 1)$ and $\big((1 - f_x) + f_x\big)\big((1 - f_y) + f_y\big) = 1$. (iii) At $f_x = f_y = 0$ only TL has non-zero weight. On a vertical cell border ($f_x \to 1$ from the left, or $f_x = 0$ from the right) both formulas reduce to linear interpolation between the same two pixels. (iv) The formula is of that form; four coefficients are determined by four values, and the system is invertible. □

Validity domain. The formula uses columns x_0 and $x_0 + 1$ and rows y_0 and $y_0 + 1$. For a template of width W and height H_t (columns $0, \ldots, W-1$, rows $0, \ldots, H_t-1$), all four neighbours exist iff

$$0 \le c_x < W - 1 \qquad \text{and} \qquad 0 \le c_y < H_t - 1.$$

Positions outside this domain correspond to scene pixels where the template is not visible (or to a thin border that one simply ignores).

Pitfall (Implementation details that cause silent errors). The following mistakes produce wrong images without raising any error.

- **Indexing order.** Arrays are indexed `image[row, column]`=`image[y, x]`.
- **Integer overflow.** Arithmetic on `uint8` values wraps around modulo 256 (e.g. $200 + 100$ gives 44). Convert to floating point before weighting and summing.
- **Casting back.** `astype(np.uint8)` truncates towards zero; rounding first (`np.rint`) is more accurate. Thanks to Proposition 9.3(ii) the result is already in $[0, 255]$, but clipping does not hurt.
- **Floor of negative numbers.** `int(-0.3)` is 0, whereas $\lfloor -0.3 \rfloor = -1$; check the bounds on the real-valued coordinates, not on truncated ones.

9.3 Alpha compositing

Definition 9.4 ("Over" compositing)**.** Given a foreground image F, a background B and an opacity map $\alpha(\mathbf{x}) \in [0, 1]$, the composite is

$$C(\mathbf{x}) = \alpha(\mathbf{x})\, F(\mathbf{x}) + \big(1 - \alpha(\mathbf{x})\big)\, B(\mathbf{x}).$$

With a binary mask ($\alpha \in \{0, 1\}$), C takes the foreground where the warped template is defined and the scene elsewhere; this is what the last line of the notebook's function does, using a mask of shape $(h, w, 1)$ that broadcasts against the three colour channels. A fractional α near the border of the warped region (e.g. the fraction of the pixel covered by the template) would remove the jagged edges, but it is not required.

Remark 9.5 (Checking your result). OpenCV's `cv.warpPerspective(src, M, (width, height))` performs the same inverse warping with bilinear interpolation. By default it interprets `M` as the map from `src` to the output and inverts it internally; with the flag `cv.WARP_INVERSE_MAP` it uses `M` directly as the output-to-source map. It is a convenient way to *check* a hand-written implementation, but the assignment asks you to write the interpolation yourself.

Remark 9.6 (Speed). The notebook loops over pixels in Python for clarity. The same computation can be vectorised by building the arrays of all pixel coordinates at once (`np.meshgrid`), transforming them with a single matrix product, and gathering the four neighbours with integer array indexing.

10 Large-scale retrieval with bags of visual words

10.1 Why pairwise matching does not scale

The matching pipeline of Part 1 compares every descriptor of the query with every descriptor of a database image. For a database of N images with n features each, a query costs $O(Nn^2d)$ operations and the database occupies Nnd numbers: with $N = 10^6$, $n = 2000$, $d = 128$ bytes per descriptor this is about 256 GB of descriptors, and $4 \cdot 10^{12}$ descriptor comparisons per query. Image retrieval systems therefore replace the set of descriptors of each image by a compact, *global* representation that can be compared quickly, and keep only a light-weight description (positions and word indices) for a final geometric check of a short list of candidates. The model used here is borrowed from text retrieval [6].

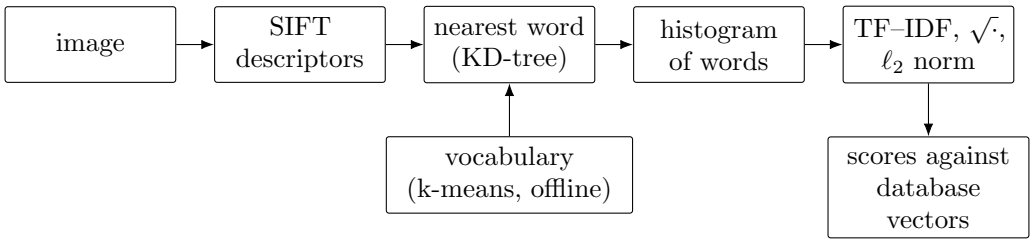


Figure 16: From an image to its bag-of-visual-words vector.

10.2 Visual vocabulary: k-means clustering

Definition 10.1 (k-means)**.** Given points $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and an integer K, k-means seeks centres $\mathbf{c}_1, \dots, \mathbf{c}_K$ and an assignment $a : \{1, \dots, n\} \to \{1, \dots, K\}$ minimising

$$J(\mathbf{c}, a) = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \mathbf{c}_{a(i)} \right\|^2 .$$

Algorithm 2 Lloyd's algorithm for k-means

1: initialise $\mathbf{c}_1, \dots, \mathbf{c}_K$ (e.g. K random data points, or k-means++)
2: **repeat**
3: **assignment:** $a(i) \leftarrow \arg\min_k \|\mathbf{x}_i - \mathbf{c}_k\|$ for every i
4: **update:** $\mathbf{c}_k \leftarrow$ mean of $\{\mathbf{x}_i : a(i) = k\}$ for every k
5: **until** the assignments do not change

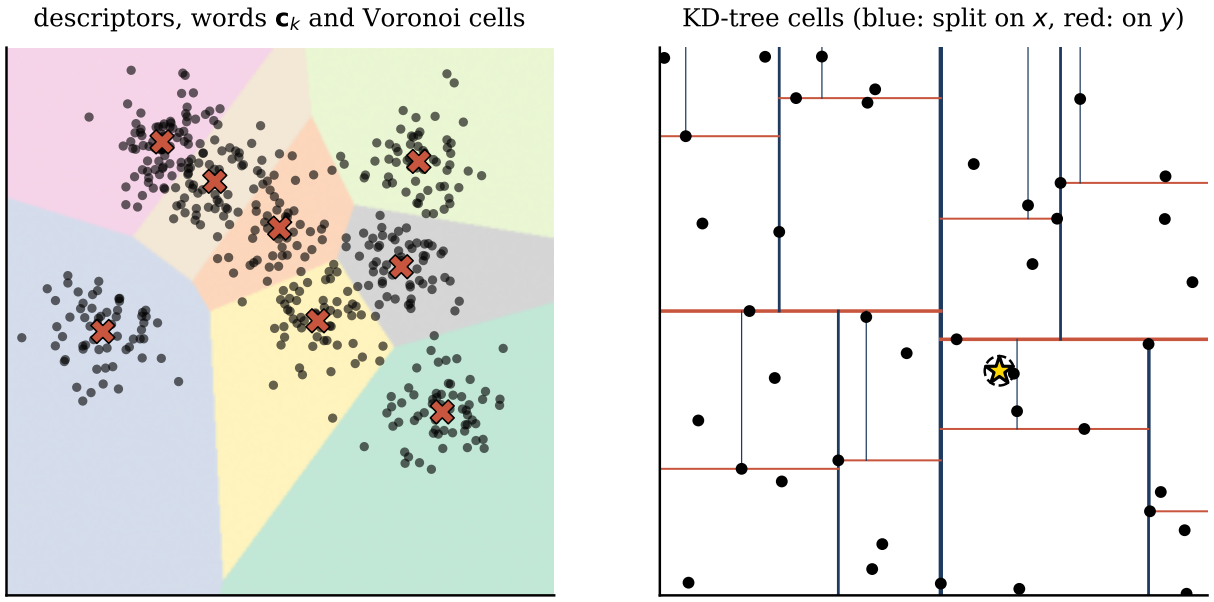
Proposition 10.2. *Each step of Lloyd's algorithm does not increase J; the algorithm terminates after finitely many iterations, at a local (not necessarily global) minimum.*

Proof. For fixed centres, assigning every point to its nearest centre minimises each term of J. For fixed assignments, $\sum_{i \in C} \|\mathbf{x}_i - \mathbf{c}\|^2$ is a strictly convex quadratic in $\mathbf{c}$ whose gradient $2\sum_{i \in C}(\mathbf{c} - \mathbf{x}_i)$ vanishes at the mean of the cluster. So J is non-increasing; it takes finitely many values (one per assignment), and the algorithm stops when an assignment repeats. Different initialisations can lead to different local minima. □

Once the centres are learned (offline, on descriptors from many images), each centre $\mathbf{c}_k$ is a *visual word* and the set of centres is the *vocabulary*. A descriptor is *quantised* to its nearest word,

$$q(\mathbf{d}) = \arg\min_{1 \le k \le K} \|\mathbf{d} - \mathbf{c}_k\| ,$$

i.e. to the *Voronoi cell* that contains it (Fig. 17a). Two descriptors are then considered to match if they fall in the same cell.



(a) Quantisation by k-means (2D illustration). **(b)** KD-tree partition and a nearest-neighbour query.

Figure 17: Left: each descriptor is replaced by the index of the Voronoi cell (visual word) it falls into. Right: a KD-tree recursively splits the space by axis-aligned hyperplanes; the star is a query, the dashed circle has radius equal to the distance to its nearest neighbour, and only cells that intersect this circle need to be examined.

Choice of K. A small vocabulary has large cells: descriptors of different structures fall in the same word, and matches by word are not discriminative. A very large vocabulary has small cells: two descriptors of the same physical point, perturbed by noise and viewpoint, may fall into neighbouring cells and the match is lost (quantisation error); the vocabulary is also more expensive to learn and to search. Retrieval systems use from 10^4 to 10^6 words; the dataset of the assignment uses $K = 10^4$.

Remark 10.3 (Consistency). The query must be described exactly like the database: same type of descriptor, same scaling convention, same vocabulary. OpenCV and VLFeat (used to build the database of the assignment) both store SIFT entries scaled by 512 in the range $[0, 255]$, which makes them compatible up to small implementation differences.

10.3 Fast nearest-word search: KD-trees

Quantising n query descriptors by brute force costs $O(nKd)$. A KD-tree [11] organises the K words so that nearest-neighbour queries can skip most of them.

Definition 10.4 (KD-tree)**.** A KD-tree on a finite set $P \subset \mathbb{R}^d$ is a binary tree built recursively: if $|P|$ is at most a *leaf size*, store P in a leaf; otherwise choose a coordinate j (e.g. the one of largest spread) and a split value v (e.g. the median of the j-th coordinates), and build the left and right subtrees on $\{p \in P : p_j \le v\}$ and $\{p \in P : p_j > v\}$. Each node corresponds to an axis-aligned box of $\mathbb{R}^d$.

With median splits the tree is balanced, has depth $O(\log K)$, and is built in $O(K \log K)$ time.

Proposition 10.5. *Algorithm 3 returns the exact nearest neighbour.*

Proof. The only discarded subtrees are "far" sides with $|q_j - v| \ge \delta^\star$. Every point p on that side satisfies $\|\mathbf{q} - p\| \ge |q_j - p_j| \ge |q_j - v| \ge \delta^\star$, so it cannot be strictly closer than the current best. □

Algorithm 3 Exact nearest-neighbour search in a KD-tree

```
function SEARCH(node, q)          ▷ global: best point p* and distance δ*, initially ∞
    if node is a leaf then update (p*, δ*) with the points of the leaf; return
    let (j, v) be the split of the node; near ← side containing q, far ← other side
    SEARCH(near, q)
    if |q_j − v| < δ* then SEARCH(far, q)
                                                                  ▷ pruning test
```

Complexity and the curse of dimensionality. In low dimension the descent reaches a leaf in $O(\log K)$ steps and few far branches survive the pruning test, so a query costs about $O(\log K)$. In high dimension this breaks down: a tree with $K = 10^4$ words has depth about 13, so along any root-to-leaf path at most 13 of the 128 coordinates are ever split, and the distance to a splitting hyperplane is a poor lower bound on the distance to the points behind it. Many branches must be explored, and the exact search can approach the cost of brute force. Large systems therefore use *approximate* search: limiting the number of visited leaves (best-bin-first), forests of randomised KD-trees (FLANN [12]), or a hierarchical k-means vocabulary [8]. For the vocabulary of the assignment an exact KD-tree is perfectly adequate.

The scikit-learn interface. `sklearn.neighbors.KDTree(X, leaf_size=40)` builds the tree on the rows of `X` (shape `(n_points, d)`). The method `tree.query(Y, k=1)` returns a pair `(dist, ind)` of arrays of shape `(n_queries, k)`: the Euclidean distances to and the row indices (in `X`) of the k nearest points of each row of `Y`. Note the two-dimensional shape even when $k = 1$.

```
import numpy as np
from sklearn.neighbors import KDTree

rng = np.random.default_rng(0)
reference = rng.uniform(0, 1, size=(1000, 3))   # 1000 points in R^3
queries = rng.uniform(0, 1, size=(5, 3))

tree = KDTree(reference)                        # built once
dist, ind = tree.query(queries, k=2)            # 2 nearest neighbours
print(dist.shape, ind.shape)                    # (5, 2) (5, 2)
print(ind[:, 0])                                # nearest point, per query

# sanity check against brute force
brute = np.argmin(((queries[:, None, :] - reference[None]) ** 2).sum(-1), axis=1)
assert np.array_equal(brute, ind[:, 0])
```

10.4 The bag-of-visual-words histogram

Definition 10.6 (BoVW histogram)**.** Let an image have descriptors $\mathbf{d}_1, \dots, \mathbf{d}_n$ with words $w_i = q(\mathbf{d}_i) \in \{1, \dots, K\}$. Its bag-of-visual-words vector is $\mathbf{h} \in \mathbb{N}^K$ with

$$h_k = \#\{i : \ w_i = k\}, \qquad k = 1, \dots, K.$$

Some properties:

- $\mathbf{h}$ has length K whatever the number of features: *all* words must have an entry, including those that do not occur in the image (value 0). Since an image has a few thousand features and $K = 10^4$, $\mathbf{h}$ is *sparse*.
- It discards the positions of the features (hence "bag"): two images with the same words in a different spatial arrangement have the same vector. Geometry is recovered later by verification (Section 11).

- It is invariant to anything the descriptors are invariant to, and robust to occlusion and clutter, which only add or remove some counts.

NumPy offers vectorised counting routines for this kind of task (for instance `np.bincount`, `np.unique(..., return_counts=True)` or `np.histogram`); read their documentation carefully, in particular how each one handles values that never occur and how it defines its bins, and make sure word indices and vocabulary positions use the same (zero-based) convention.

10.5 TF–IDF weighting and normalisation

Not all words are equally informative. A word that appears in almost every image (a generic texture) says little about which image we are looking at, while a rare word is very discriminative. This is measured by the *inverse document frequency*.

Definition 10.7 (TF–IDF)**.** Let the database contain N images and let N_k be the number of images that contain word k at least once. For an image with counts $\mathbf{h}$ and $n = \sum_k h_k$ features,

$$\mathrm{tf}_k = \frac{h_k}{n}, \qquad \mathrm{idf}_k = \log \frac{N}{N_k}, \qquad v_k = \mathrm{tf}_k \cdot \mathrm{idf}_k \,.$$

A word present in every image gets weight $\log 1 = 0$ and is effectively ignored (a "stop word"). In the notebook the `idf` vector is precomputed on the database and the raw counts are multiplied by it; the normalisation by n is unnecessary because of the following fact.

Proposition 10.8 (Global scale does not matter)**.** *Let* $\Phi(\mathbf{v}) = \sqrt{\mathbf{v}}/\left\|\sqrt{\mathbf{v}}\right\|_2$ *(square root taken entry-wise) for* $\mathbf{v} \geq 0$*,* $\mathbf{v} \neq \mathbf{0}$*. Then* $\Phi(\lambda \mathbf{v}) = \Phi(\mathbf{v})$ *for every* $\lambda > 0$*.*

Proof. $\sqrt{\lambda \mathbf{v}} = \sqrt{\lambda}\sqrt{\mathbf{v}}$ and the factor $\sqrt{\lambda}$ cancels in the normalisation. □

The square root (Hellinger normalisation). The square root compresses large counts. This counters *burstiness* [10]: repetitive structures (windows of a facade, a brick wall, text) produce the same word many times, and without compression such a word would dominate the similarity. There is also a clean interpretation.

Proposition 10.9 (Square root and ℓ_2 normalisation equal Hellinger)**.** *For* $\mathbf{v} \geq 0$*,* $\mathbf{v} \neq \mathbf{0}$*,*

$$\frac{\sqrt{\mathbf{v}}}{\left\|\sqrt{\mathbf{v}}\right\|_2} = \sqrt{\frac{\mathbf{v}}{\|\mathbf{v}\|_1}}.$$

Consequently, for two such vectors, $\langle \Phi(\mathbf{v}), \Phi(\mathbf{v}') \rangle = \sum_k \sqrt{p_k p'_k}$ *where* $\mathbf{p} = \mathbf{v}/\|\mathbf{v}\|_1$ *and* $\mathbf{p}' = \mathbf{v}'/\|\mathbf{v}'\|_1$ *are probability vectors. This is the* Bhattacharyya coefficient*; it lies in* $[0, 1]$ *and equals* 1 *iff* $\mathbf{p} = \mathbf{p}'$*.*

Proof. $\left\|\sqrt{\mathbf{v}}\right\|_2^2 = \sum_k v_k = \|\mathbf{v}\|_1$ because $\mathbf{v} \geq 0$. The bound follows from Cauchy–Schwarz: $\sum_k \sqrt{p_k}\sqrt{p'_k} \leq \left(\sum_k p_k\right)^{1/2}\left(\sum_k p'_k\right)^{1/2} = 1$, with equality iff $\sqrt{\mathbf{p}}$ and $\sqrt{\mathbf{p}'}$ are proportional, i.e. equal. □

10.6 Similarity scores

Definition 10.10 (Cosine similarity)**.** For non-zero $\mathbf{a}, \mathbf{b}$, $\cos(\mathbf{a}, \mathbf{b}) = \dfrac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\|\,\|\mathbf{b}\|}$. For ℓ_2-normalised vectors it reduces to the dot product.

Proposition 10.11 (Dot product and Euclidean distance rank identically)**.** *If* $\|\mathbf{q}\| = \|\mathbf{v}_j\| = 1$ *for all* j*, then* $\|\mathbf{q} - \mathbf{v}_j\|^2 = 2 - 2\langle \mathbf{q}, \mathbf{v}_j \rangle$*. Hence sorting database images by decreasing dot product with the query is the same as sorting them by increasing Euclidean distance.*

Proof. Proposition 5.3 with $\|\mathbf{q}\|^2 = \|\mathbf{v}_j\|^2 = 1$. □

The retrieval score of database image j is therefore $s_j = \langle \mathbf{q}, \mathbf{v}_j \rangle \in [0, 1]$ (1 for identical word distributions, 0 when no word is shared), where $\mathbf{q}$ and $\mathbf{v}_j$ are the processed (TF–IDF, square root, ℓ_2) vectors. All N scores can be obtained at once with a single matrix–vector product; before writing it, check the shapes of the arrays you are given (which axis indexes words and which indexes images) and the type of the database matrix.

Remark 10.12 (Sparse matrices). `scipy.io.loadmat` returns MATLAB sparse arrays as `scipy.sparse` matrices. They support `@`, `.T`, `.shape` and `.toarray()`, but products of sparse matrices with dense arrays may return `np.matrix` objects or two-dimensional arrays; use `np.asarray(...)` and `.ravel()` to obtain a plain one-dimensional vector when you need one.

10.7 Inverted files

Because the vectors are sparse, the dot products need not be computed by scanning all K entries of all N images. An *inverted file* stores, for each word k, the list of images in which it appears together with the weight $v_{j,k}$. To score a query, one walks only through the lists of the words present in the query and accumulates $q_k v_{j,k}$ into the score of each image encountered. The cost is proportional to the total length of the visited lists, which is small when the vocabulary is large and common words are down-weighted (or removed). This is the same structure that makes text search engines fast, and it is what a sparse matrix product does implicitly.

10.8 Evaluating a ranking

For a query with a set of relevant database images, the *precision at* k is the fraction of relevant images among the top k; the *average precision* (AP) averages the precision at the ranks where relevant images appear, and the mean of AP over queries (mAP) is the standard retrieval metric. When, as in the assignment, a query has only one (or very few) relevant images, one simply looks at the rank of the correct image and at the gap between its score and the scores of the others.

11 Geometric re-ranking of a shortlist

11.1 Why re-rank?

The BoVW score ignores where the words are. Two unrelated images can share many words by chance (common textures, similar brush strokes), while the correct image may lose a few points because of occlusion or illumination. The standard remedy [7] is to keep the fast BoVW ranking to produce a *shortlist* of the top candidates (say 20 to 100), and to re-rank this shortlist with a *spatial verification* score: the number of matches that are consistent with a geometric transformation between the query and the candidate. Only the frames and their word indices are needed, not the descriptors.

11.2 Tentative matches from visual words

Definition 11.1. Given the words $w_1, \dots, w_{n_1}$ of the query features and $w'_1, \dots, w'_{n_2}$ of a database image, the *word-based tentative matches* are the pairs (i, j) with $w_i = w'_j$.

This replaces nearest-neighbour search by a hash lookup, but it has its own characteristics:

- The relation is *many-to-many*: if word k occurs a_k times in the query and b_k times in the candidate, it produces $a_k b_k$ pairs, of which at most $\min(a_k, b_k)$ can be correct. Bursty words can create thousands of pairs; implementations therefore cap the number of pairs per word (the notebook's `max_pairs_per_word`) or ignore very frequent words.

- Sharing a word is a much weaker condition than being mutual nearest neighbours: a Voronoi cell of a 10^4-word vocabulary in $\mathbb{R}^{128}$ contains many descriptors of different structures, and there is no equivalent of the ratio test. Many tentative matches are therefore wrong, which is precisely why the geometric check is needed.

11.3 A simpler transformation model

For the paintings of the assignment, the photographs are taken roughly upright, so the rotation between a query and its database image is close to zero. One can then use the model

$$\mathbf{x}_2 = s\,\mathbf{x}_1 + \mathbf{t}, \qquad s > 0,\ \mathbf{t} \in \mathbb{R}^2,$$

a similarity *without rotation*, with 3 degrees of freedom (isotropic scaling and translation). A more constrained model has two advantages: hypotheses are estimated more reliably, and it is harder for wrong matches to agree with a model by chance, so that a wrong candidate collects fewer accidental inliers and the gap with the correct one widens. The verification proceeds as in Section 8: hypothesise from a minimal sample of tentative matches, count the matches whose transfer error is below a threshold, keep the best count as the score of the candidate, and sort the shortlist by decreasing score.

Remark 11.2 (Hough voting). Since each frame correspondence alone proposes a full hypothesis, an alternative to random sampling is to let *every* tentative match vote in a discretised space of transformation parameters (for example $(\log s, t_x, t_y)$) and look for the densest bin. This is the approach used by Lowe [1] before a final least-squares verification.

Pitfall (Mixing frame conventions). The query frames are computed with OpenCV in the notebook, whereas the database frames were computed offline (with VLFeat, in MATLAB). Before deriving a scale from a ratio of frame scales, make sure both use the same definition of "scale"; a constant factor between the two conventions would bias *every* hypothesis. Also remember the MATLAB conventions listed in Appendix A.

11.4 What to look for in the analysis

The interesting quantity is the *separation* between the score of the correct painting and the scores of the others, compared with the separation obtained by the BoVW score alone. A large gap in the number of inliers indicates that spatial consistency is a much more discriminative signal than word co-occurrence; it also allows one to set a threshold for deciding whether the query is in the database at all. Think about which factors make the inlier counts of wrong candidates non-zero (random agreements, repeated words, bursty textures, the size of the threshold) and which factors can reduce the count of the correct one (perspective, a too small threshold, quantisation errors).

A NumPy and SciPy toolbox

This appendix collects the generic tools and conventions that the assignment relies on, with toy examples unrelated to the tasks.

A.1 Shapes, axes and broadcasting

- A reduction along `axis=1` of an array of shape `(n, m)` produces one value per *row* (shape `(n,)`); along `axis=0`, one value per *column*.
- Shapes `(n,)` (a 1D vector) and `(n, 1)` (a column) are different: combining them in an arithmetic operation broadcasts to `(n, n)`, which is almost never intended. Use `.ravel()` or `.reshape(-1, 1)` to make shapes explicit.

- Broadcasting aligns trailing dimensions: `(h, w, 1) * (h, w, 3)` works channel-wise, `a[:, None] - b[None, :]` builds all pairwise differences of two vectors.

A.2 Finding minima and the k smallest values

```
import numpy as np
v = np.array([7.0, 2.0, 9.0, 4.0, 1.0])

np.argmin(v)              # 4: position of the minimum
np.sort(v)[:2]            # array([1., 2.]): two smallest values, O(n log n)
np.argsort(v)[:2]         # array([4, 1]): their positions

# Partial selection in O(n): after partitioning around position k, position k holds
# the value a full sort would put there; smaller values come before it, unordered.
p = np.partition(v, 1)    # p[0] <= p[1] are the two smallest values
idx = np.argpartition(v, 1)   # same thing, returning positions
```

All these functions accept an `axis` argument and then work independently on every row (or column) of a 2D array. With `np.partition(a, k, axis=1)` only position `k` is guaranteed to be "sorted"; positions before it are smaller but unordered, so ask for the partition point you actually need.

A.3 Selecting with masks and indices

```
x = np.array([0.3, 1.7, 0.1, 2.4])
mask = x < 1.0                # array([ True, False,  True, False])
np.flatnonzero(mask)          # array([0, 2]): indices where the condition holds
x[mask]                       # array([0.3, 0.1])

table = np.arange(12).reshape(4, 3)
rows = np.array([3, 0, 0])
table[rows, :]                # rows 3, 0, 0: "fancy" indexing
table[rows, [2, 1, 0]]        # elements (3,2), (0,1), (0,0)
```

A.4 Safe division

```
num = np.array([1.0, 2.0, 3.0])
den = np.array([2.0, 0.0, 4.0])
out = np.full_like(num, fill_value=np.inf)   # value for degenerate entries
np.divide(num, den, out=out, where=den > 0)  # [0.5, inf, 0.75], no warning
```

The fill value encodes a *decision* about the degenerate case; choose it according to what the quantity is supposed to mean. `np.finfo(np.float32).eps` gives the machine epsilon if you prefer adding a small constant to the denominator.

A.5 Angles

- `np.arctan2(y, x)` takes the y component *first* and returns a value in $(-\pi, \pi]$.
- NumPy trigonometric functions use radians; `np.deg2rad` and `np.rad2deg` convert. OpenCV keypoint angles are in degrees (the notebook's helper converts them).
- To bring an angle into $(-\pi, \pi]$: `np.angle(np.exp(1j * a))` or `(a + np.pi) % (2 * np.pi) - np.pi` (the latter returns $[-\pi, \pi)$).

A.6 Random numbers

- `np.random.randint(low, high)` returns an integer in $\{\texttt{low}, \ldots, \texttt{high} - 1\}$ (the upper bound is excluded).
- `rng = np.random.default_rng(seed)` creates a reproducible generator; `rng.choice(n, size=m, replace=False)` draws m distinct indices.

A.7 Data coming from MATLAB

The preprocessed painting database is a `.mat` file produced in MATLAB with VLFeat. When you read such data in Python, keep in mind:

- **One-based indexing.** MATLAB indices start at 1. Word indices, image indices and pixel coordinates stored by MATLAB code are typically one-based, whereas Python's are zero-based. Check the range of the values (e.g. minimum and maximum against the vocabulary size) before using them as indices.
- **Orientation of arrays.** MATLAB code often stores one item per *column* (e.g. frames as a $4 \times n$ matrix, with rows x, y, s, θ), while the notebook uses one item per *row*. Inspect `.shape` and transpose when needed.
- **Nested containers.** Structures and cell arrays become nested NumPy object arrays, hence expressions such as `imdb['images'][0][0][5][0][idx]`. `np.squeeze`, `np.asarray(..., dtype=float)` and `.item()` help to unwrap them.
- **Integer types.** Indices may be stored as `uint32` or `double`; convert them to a signed integer type before subtracting or comparing.
- **Sparse matrices** become `scipy.sparse` objects (see Section 10.6).

A.8 Image types

- For ordinary images, `skimage.io.imread` returns `uint8` arrays in $[0, 255]$, whereas `resize` (from `skimage.transform`) returns floats in $[0, 1]$ unless it is called with `preserve_range=True`. Always check `dtype` and value range before arithmetic.
- PNG files may have a fourth (alpha) channel: shape `(h, w, 4)`. Drop it with `img[..., :3]` if only RGB is expected.
- `plt.imshow` expects floats in $[0, 1]$ or integers in $[0, 255]$; values outside these ranges are clipped, which may hide bugs.

References

[1] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV*, 60(2):91–110, 2004.

[2] T. Lindeberg. Feature detection with automatic scale selection. *IJCV*, 30(2):79–116, 1998.

[3] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[4] R. Hartley. In defense of the eight-point algorithm. *IEEE TPAMI*, 19(6):580–593, 1997.

[5] M. A. Fischler and R. C. Bolles. Random sample consensus. *Communications of the ACM*, 24(6):381–395, 1981.

[6] J. Sivic and A. Zisserman. Video Google: a text retrieval approach to object matching in videos. *ICCV*, 2003.

[7] J. Philbin, O. Chum, M. Isard, J. Sivic and A. Zisserman. Object retrieval with large vocabularies and fast spatial matching. *CVPR*, 2007.

[8] D. Nistér and H. Stewénius. Scalable recognition with a vocabulary tree. *CVPR*, 2006.

[9] R. Arandjelović and A. Zisserman. Three things everyone should know to improve object retrieval. *CVPR*, 2012.

[10] H. Jégou, M. Douze and C. Schmid. On the burstiness of visual elements. *CVPR*, 2009.

[11] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.

[12] M. Muja and D. G. Lowe. Fast approximate nearest neighbors with automatic algorithm configuration. *VISAPP*, 2009.

[13] K. Mikolajczyk et al. A comparison of affine region detectors. *IJCV*, 65:43–72, 2005.

[14] R. Szeliski. *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.

[15] A. Vedaldi and A. Zisserman. VGG practical: Instance-level recognition. University of Oxford.